

单位代码： 10293 密 级： _____

南京邮电大学

专业学位硕士学位论文



论文题目： Swift: 基于强化学习的多信号融合自适应网络拥塞控制算法

学 号	1224045520
姓 名	杭天铖
导 师	王俊昌教授
专业学位类别	电子信息
类 型	全日制
专 业（领域）	计算机技术
论文提交日期	2026 年 3 月

摘要

随着云计算、移动互联网、人工智能和物联网技术的快速发展，端到端网络传输正在从单一固定终端和局域高质量链路，扩展到跨地域远距离传输、蜂窝/无线/卫星等多接入网络，以及手机、笔记本、台式机和边缘设备等多模态终端并存的复杂场景。传统基于丢包的拥塞控制算法（如 TCP NewReno、CUBIC）存在拥塞信号滞后、高带宽延迟积（Bandwidth-Delay Product, BDP）网络收敛缓慢、无法区分拥塞程度等固有缺陷；而基于延迟的算法（如 TCP Vegas、BBR）则面临往返时延（Round-Trip Time, RTT）测量不稳定、与丢包算法共存时公平性差等问题。本文提出 Swift 算法——一种基于强化学习辅助的多信号融合自适应网络拥塞控制方法。该算法名称寓意“雨燕”，象征着高吞吐量（High Throughput）与低延迟（Low Latency）的设计目标，如同雨燕般轻盈敏捷、反应迅速。

Swift 算法的核心创新收敛为三点：（1）面向远距离与异构终端传输的 11 维有效 RL 观测子空间设计，该子空间嵌入于 15 元素 OpenGym Box 传输容器，另含 4 项跨进程路由用的元数据通道，将显式拥塞通知（Explicit Congestion Notification, ECN）状态、拥塞控制状态机（Congestion Algorithm State）状态、拥塞事件类型、RTT 和在途字节等协议栈信息纳入强化学习状态表示；（2）多信号融合拥塞判定与差异化响应机制，采用分层优先级机制综合分析丢包、ECN、超时和拥塞状态信号，并针对不同拥塞类型采用不同的窗口保留因子（Window Retention Factor）——丢包 0.70、ECN 标记 0.75、超时 0.50，同时引入连续缩减保护机制（最多连续 3 次缩减后保持窗口）和降窗后 4 个 ACK 事件冻结机制，以区分拥塞信号的性质和严重程度并抑制窗口快速反弹；（3）强化学习辅助的自适应融合控制策略，根据 RTT 膨胀比、奖励信号的指数移动平均（Exponential Moving Average, EMA）、连续增长趋势等多因素动态调整乘性增加因子 α （取值范围 [0.85, 1.30]，基准值 1.10），并采用窗口化最大值过滤（Windowed Max-filter）估计 BDP，在拥塞避免阶段围绕 $\alpha \times BDP$ 目标窗口进行有界步长的渐进拉升或回落。

本文基于 ns-3 网络仿真平台和 ns3-gym/OpenGym 强化学习框架实现了 Swift 算法，采用 ZeroMQ 进程间通信实现仿真进程与决策进程的同步协作。在覆盖近端高速、无线接入、5G 移动网络、城域广域网（Wide Area Network, WAN）和卫星通信等 21 种场景、84 组实验中，经数据质量审计筛选出 9 种具备全部四种协议完整对比数据的主要场景；为进一步评估算法在多模态终端和共享链路中面对突发性非响应式跨流量时的鲁棒性，仿真器在哑铃拓扑上额外引入 enable_udp_burst 控制开关，构建了“纯 TCP”与“TCP + 800 Mbps UDP Burst”的 A/B 严格对照实验矩阵。稳态实验结果表明：在近端高速场景（10G/25G/20G），Swift 保持与 NewReno/CUBIC 相当的吞吐量（差距 <0.01%），同时将排队延迟降低 83%~93%（从 1.9~2.3ms 降至 0.14~0.36ms）；在中速网络场景（5G eMBB、城域 WAN、WiFi AC/AX），延迟

降低 9%~24% 且全部实现零丢包；在 GEO 卫星长距离场景中，凭借零丢包特性实现吞吐量相对 NewReno/CUBIC 均值提升 28.8%。**跨流量鲁棒性实验**进一步表明：在全部 17 个完整对照场景中，Swift 是唯一维持零丢包率的算法（CUBIC 最大新增丢包率 +3.45%、BBR +1.58%）；Swift 的吞吐量回落幅度最小（均值仅 19.78%，BBR 为崩塌式 55.55%）；且近端高速场景的延迟降幅被完整保留（S1~S3 仍为 -82.28%/-92.91%/-91.02%），GEO 卫星场景中 Swift 的吞吐量优势由 +28.8% 进一步放大至 +86.8%。本文同时如实披露了 Swift 在中低速和中等 RTT 场景中存在小幅度的延迟抬升，以及 WAN 场景吞吐量仍有进一步上探空间，其机理在于 $\alpha \times BDP$ 窗口目标机制在高 RTT 下的扩张幅度仍可进一步收紧，相应的算法层改进方向已在第 5 章给出。

关键词：拥塞控制，强化学习，多信号融合，显式拥塞通知，自适应参数调优，远距离传输，异构终端

Abstract

With the rapid development of cloud computing, mobile Internet, artificial intelligence, and Internet-of-Things technologies, end-to-end network transmission is expanding from fixed terminals and local high-quality links to long-distance cross-region paths, cellular/wireless/satellite access networks, and heterogeneous endpoint environments involving phones, laptops, desktops, edge devices, and other terminal classes. Traditional loss-based congestion control algorithms (such as TCP NewReno and CUBIC) suffer from delayed congestion signals, slow convergence in high Bandwidth-Delay Product (BDP) networks, and inability to distinguish congestion severity. Meanwhile, delay-based algorithms (such as TCP Vegas and BBR) face unstable Round-Trip Time (RTT) measurements and poor fairness when coexisting with loss-based algorithms.

This paper proposes Swift, a reinforcement-learning-assisted multi-signal fusion adaptive network congestion control algorithm. The name “Swift” symbolizes the design goals of High Throughput and Low Latency, like a swift that is light, agile, and responsive. The core innovations of Swift are consolidated into three aspects: (1) an 11-dimensional effective RL observation subspace for long-distance and heterogeneous endpoint transmission, embedded in a 15-element OpenGym Box transport container with four additional metadata channels for inter-process routing, which incorporates Explicit Congestion Notification (ECN) status, Congestion Algorithm (CA) state, congestion event types, RTT, and bytes in flight into the reinforcement learning state representation; (2) a multi-signal congestion determination and differentiated response mechanism, which hierarchically analyzes packet loss, ECN, timeout, and congestion-state signals and applies different window retention factors for different congestion types (packet loss: 0.70, ECN: 0.75, timeout: 0.50), together with consecutive decrease protection and a four-ACK post-decrease freeze mechanism; and (3) a reinforcement-learning-assisted adaptive fusion control strategy, which dynamically adjusts the multiplicative increase factor $\alpha \in [0.85, 1.30]$ based on RTT inflation gradients, reward EMA, and consecutive growth trends, estimates BDP through a windowed max-filter, and moves the congestion window toward the $\alpha \times BDP$ target with bounded pull-up or drain steps during congestion avoidance.

We implemented Swift based on the ns-3 network simulation platform and ns3-gym/OpenGym reinforcement learning framework, with ZeroMQ inter-process communication for synchronous interaction between the simulation process and the decision process. Extensive experiments across 21 scenarios covering near-end high-speed links, wireless access, 5G, metropolitan/WAN, and satellite communications (84 experiment groups) yielded 9 primary scenarios with complete four-protocol com-

parison data after data quality auditing; to further evaluate robustness against bursty non-responsive cross-traffic in heterogeneous endpoint and shared-link environments, the simulator additionally exposes an `enable_udp_burst` switch that constructs a strict A/B experimental matrix between “pure TCP” and “TCP + 800 Mbps UDP Burst” settings on the same dumbbell topology. **Steady-state results** demonstrate that: in near-end high-speed scenarios (10G/25G/20G), Swift maintains throughput comparable to NewReno/CUBIC (gap < 0.01%) while reducing queuing latency by 83%–93% (from 1.9–2.3 ms to 0.14–0.36 ms); in medium-speed scenarios (5G eMBB, metro WAN, WiFi AC/AX), latency is reduced by 9%–24% with zero packet loss; in GEO satellite long-distance scenarios, zero-loss behavior yields a 28.8% throughput improvement over the NewReno/CUBIC mean. **Cross-traffic robustness experiments** further show that across all 17 complete comparison scenarios, Swift is the *only* algorithm to maintain zero packet loss (CUBIC adds up to +3.45% loss, BBR up to +1.58%); Swift’s throughput degradation is the smallest (mean 19.78%, compared with BBR’s collapse of 55.55%); and the near-end high-speed latency advantage is fully preserved (−82.28%/−92.91%/−91.02% for S1–S3), while Swift’s GEO satellite throughput advantage expands from +28.8% to +86.8%. Experiments also faithfully report that Swift exhibits a mild latency uplift in medium-to-low-speed and medium-RTT scenarios, and that there remains room for further throughput exploration in WAN, attributable to the expansion magnitude of the $\alpha \times BDP$ window target under high RTT, with corresponding algorithm-level refinement directions presented in Chapter 5.

Key Words: Congestion Control, Reinforcement Learning, Multi-signal Fusion, Explicit Congestion Notification, Adaptive Parameter Tuning, Long-distance Transmission, Heterogeneous Endpoints

目 录

摘要	I
Abstract	III
插图索引	IX
表格索引	X
专用术语注释表	XI
第一章 绪论	1
1.1 研究背景与动机	1
1.2 研究目标与主要贡献	2
1.3 论文组织结构	3
第二章 相关工作	4
2.1 传统拥塞控制算法	4
2.1.1 基于丢包的拥塞控制算法	4
2.1.2 基于延迟的拥塞控制算法	6
2.1.3 混合拥塞控制算法	7
2.2 显式拥塞通知机制	8
2.2.1 ECN 工作原理	8
2.2.2 ECN 相关拥塞控制算法	8
2.3 基于机器学习的拥塞控制方法	10
2.3.1 监督学习方法	10
2.3.2 强化学习方法	10
2.4 远距离与异构终端网络特性分析	11
2.4.1 路径与接入特性	11
2.4.2 流量特性分析	12
2.4.3 应用延迟需求分析	12
2.4.4 网络设备与反馈机制特性	12
2.5 拥塞控制的理论基础	13
2.5.1 网络流量模型	13
2.5.2 排队论分析	14
2.5.3 控制论视角	14
2.6 强化学习在网络中的应用综述	15
2.6.1 路由优化	15
2.6.2 资源分配	16
2.6.3 安全防护	16
2.6.4 与 Swift 的关系	16
2.7 本章小结	17
第三章 Swift 算法设计与实现	18
3.1 系统架构概述	18
3.2 强化学习问题建模	19
3.2.1 状态空间设计理论	20
3.2.2 动作空间设计理论	20
3.2.3 奖励函数设计理论	21
3.2.4 策略表示理论	21

3.3	11 维观测空间详细设计	21
3.3.1	连接标识参数 (索引 0、3)	22
3.3.2	时间信息参数 (索引 1、2)	22
3.3.3	窗口状态参数 (索引 4、5)	23
3.3.4	传输性能参数 (索引 6、7、8)	23
3.3.5	延迟测量参数 (索引 9、10)	23
3.3.6	回调类型参数 (索引 11)	24
3.3.7	拥塞控制状态参数 (索引 12、13、14)	24
3.4	多信号融合拥塞检测算法	25
3.4.1	拥塞信号分类	25
3.4.2	分层优先级机制	26
3.4.3	信号优先级与误判抑制策略	26
3.4.4	完整拥塞检测算法	27
3.5	差异化拥塞响应机制	27
3.5.1	窗口保留因子设计	28
3.5.2	差异化响应的理论依据	28
3.5.3	慢启动阈值更新	29
3.5.4	差异化拥塞响应算法	29
3.6	强化学习驱动的自适应参数调优	29
3.6.1	乘性增加因子的作用	29
3.6.2	多因素调整机制	30
3.6.3	参数边界约束	31
3.6.4	自适应参数调优算法	33
3.7	融合控制策略	33
3.7.1	带宽延迟积估计	33
3.7.2	慢启动阶段策略	33
3.7.3	拥塞避免阶段策略	34
3.7.4	窗口边界约束	34
3.7.5	融合控制策略算法	35
3.8	每流独立状态管理	35
3.8.1	状态数据结构	35
3.8.2	状态查找与更新	35
3.8.3	状态生命周期管理	37
3.9	实现细节与优化技术	37
3.9.1	系统架构设计	38
3.9.2	核心算法伪代码	38
3.9.3	数据结构设计	38
3.9.4	性能优化技术	38
3.9.5	OpenGym/ZeroMQ 同步交互路径	40
3.9.6	正确性保证机制	40
3.10	本章小结	40

第四章 实验评估	42
4.1 实验环境配置	42
4.1.1 硬件环境	42
4.1.2 软件环境	42
4.1.3 网络拓扑	42
4.1.4 仿真参数	43
4.1.5 UDP Burst 干扰流量与实验矩阵	43
4.2 评测指标	44
4.2.1 吞吐量 (Throughput)	44
4.2.2 延迟 (Latency)	44
4.2.3 丢包率 (Loss Rate)	45
4.2.4 公平性 (Fairness)	45
4.3 对比算法	45
4.3.1 TCP NewReno	45
4.3.2 TCP CUBIC	45
4.3.3 TCP BBR	45
4.4 实验场景设计	46
4.4.1 场景覆盖范围	46
4.4.2 9 种主要实验场景	46
4.4.3 数据采集与质量控制	47
4.5 实验方法论	47
4.5.1 实验设计原则	47
4.5.2 统计分析方法	47
4.6 实验结果分析	47
4.6.1 吞吐量对比	47
4.6.2 延迟对比	48
4.6.3 丢包率对比	49
4.6.4 延迟抖动对比	49
4.7 深入分析	50
4.7.1 场景分类讨论	50
4.7.2 机制有效性分析	51
4.7.3 参数机制分析	52
4.7.4 补充场景分析	52
4.8 UDP Burst 跨流量鲁棒性评估	52
4.8.1 聚合鲁棒性指标	53
4.8.2 头部场景逐一验证	53
4.8.3 无线接入与卫星信道下的零丢包鲁棒性补充	55
4.8.4 鲁棒性机理分析	56
4.8.5 算法优势与适用边界总结	56
4.9 本章小结	56
第五章 讨论	57
5.1 设计考量	57
5.1.1 11 维观测空间的设计原理	57
5.1.2 差异化窗口保留因子的选择	57
5.1.3 自适应参数调优的设计	57

5.2	理论分析	58
5.2.1	收敛性分析	58
5.2.2	公平性分析	58
5.2.3	复杂度分析	58
5.3	适用边界与改进方向	58
5.3.1	中低速与中等 RTT 延迟抬升	58
5.3.2	广域网吞吐量进一步上探空间	59
5.3.3	参数自动调优	59
5.3.4	深度强化学习增强	59
5.3.5	真实部署验证	59
5.4	与相关工作的比较	60
5.4.1	与 Aurora 的比较	60
5.4.2	与 DCTCP 的比较	60
5.4.3	与 BBR 的比较	60
5.4.4	与 PCC 的比较	60
5.5	敏感性分析	61
5.5.1	窗口保留因子的理论依据	61
5.5.2	连续递减保护阈值的数学分析	61
5.5.3	RTT 膨胀比阈值的设计考量	62
5.6	计算开销分析	62
5.6.1	时间复杂度	62
5.6.2	空间复杂度	62
5.6.3	工程开销来源	62
5.7	理论分析深入探讨	63
5.7.1	多信号融合的信息论基础	63
5.7.2	差异化响应的最优控制分析	63
5.7.3	自适应参数调优的强化学习视角	64
5.7.4	收敛性的 Lyapunov 分析	64
5.7.5	公平性的博弈论分析	64
5.8	本章小结	65
第六章	结论与展望	66
6.1	研究总结	66
6.2	研究展望	68
6.2.1	中低速与中等 RTT 延迟抬升、广域网吞吐量进一步上探	68
6.2.2	策略表达能力扩展	69
6.2.3	多目标优化扩展	69
6.2.4	真实网络部署与验证	69
6.2.5	安全性、鲁棒性与跨层协同	69
6.2.6	异构网络与新兴传输协议适配	69
6.2.7	可解释性与能效扩展	70
6.3	应用场景展望	70
6.4	研究意义与社会价值	70

参考文献 72

附录 A 程序清单 74

附录 B 攻读专业学位硕士期间撰写的论文 75

附录 C 攻读专业学位硕士期间申请的专利 76

附录 D 攻读专业学位硕士期间参加的科研项目 77

致谢 78

插图索引

图 1.1	Swift 算法核心贡献概览	2
图 2.1	拥塞控制算法分类体系	4
图 2.2	ECN 信号传递工作流程	9
图 3.1	Swift 系统架构	18
图 4.1	哑铃型 (Dumbbell) 实验网络拓扑	43

表格索引

表 2.1	代表性拥塞控制算法综合对比	16
表 3.1	Swift 11 维观测空间参数定义	22
表 3.2	差异化窗口保留因子	28
表 4.1	默认仿真参数	43
表 4.2	9 种主要实验场景配置	46
表 4.3	聚合吞吐量对比 (Mbps)	48
表 4.4	平均端到端延迟对比 (ms)	48
表 4.5	丢包率对比 (%)	49
表 4.6	延迟抖动对比 (ms)	50
表 4.7	UDP Burst 跨流量鲁棒性聚合指标 ($n = 17$)	53
表 4.8	主要场景在 UDP Burst 对照下的性能 (无干扰 / 有干扰)	54
表 4.9	Swift 算法优势与适用边界	56
表 5.1	Swift 与 Aurora 的详细对比	60
表 5.2	Swift 与 DCTCP 的详细对比	60
表 5.3	Swift 与 BBR 的详细对比	61

专用术语注释表

符号说明

$cwnd$	拥塞窗口 (Congestion Window), TCP 发送端在未收到确认的情况下最多可以发送的数据量 (字节)
$ssthresh$	慢启动阈值 (Slow Start Threshold), 区分慢启动和拥塞避免阶段的临界值 (字节)
α	乘性增加因子 (Multiplicative Increase Factor), 控制拥塞避免阶段窗口增长速度, 取值范围 $[0.85, 1.30]$, 基准值 1.10
γ	加性增加因子 (Additive Increase Factor), 控制窗口的加性增量, 默认值为 1.0
ρ	窗口保留因子 (Window Retention Factor), 拥塞响应后新窗口与原窗口的比值
ρ_{loss}	丢包保留因子, 默认值 0.70
ρ_{ecn}	ECN 保留因子, 默认值 0.75
$\rho_{timeout}$	超时保留因子, 默认值 0.50
η	奖励信号指数移动平均 (EMA) 衰减系数, 默认值 0.1
BDP	带宽延迟积 (Bandwidth-Delay Product), 网络链路能够容纳的最大在途数据量 (字节)
MSS	最大报文段长度 (Maximum Segment Size), 典型值为 1460 字节
RTT	往返时延 (Round-Trip Time), 数据包从发送端到接收端再返回的总时间
$W(t)$	时刻 t 的拥塞窗口大小
W_{max}	上次拥塞事件前的最大窗口大小
K	CUBIC 算法中恢复到 W_{max} 所需的时间
C	链路容量或 CUBIC 缩放因子
β	窗口缩减因子
δ	延迟惩罚权重参数
λ	丢包惩罚权重参数
H	Hurst 参数, 描述流量自相似性

$\bar{Q}$	平均队列长度
$\bar{W}$	平均等待时间
τ	反馈延迟（约等于 RTT）

缩略词

TCP	传输控制协议 (Transmission Control Protocol)
ECN	显式拥塞通知 (Explicit Congestion Notification)
RTT	往返时延 (Round-Trip Time)
BDP	带宽延迟积 (Bandwidth-Delay Product)
MDP	马尔可夫决策过程 (Markov Decision Process)
RL	强化学习 (Reinforcement Learning)
AIMD	加性增乘性减 (Additive Increase Multiplicative Decrease)
MSS	最大报文段长度 (Maximum Segment Size)
DCTCP	数据中心 TCP (Data Center TCP)
BBR	瓶颈带宽和往返传播时间 (Bottleneck Bandwidth and Round-trip propagation time)
CUBIC	三次函数拥塞控制算法
ACK	确认 (Acknowledgment)
SACK	选择性确认 (Selective Acknowledgment)
CE	拥塞经历 (Congestion Experienced)
ECE	ECN 回显 (ECN-Echo)
CWR	拥塞窗口缩减 (Congestion Window Reduced)
RED	随机早期检测 (Random Early Detection)
CA	拥塞算法 (Congestion Algorithm)
PPO	近端策略优化 (Proximal Policy Optimization)
DQN	深度 Q 网络 (Deep Q-Network)
PCC	面向性能的拥塞控制 (Performance-oriented Congestion Control)
ZMQ	ZeroMQ 消息队列
SDN	软件定义网络 (Software Defined Networking)
QoS	服务质量 (Quality of Service)

QUIC	快速 UDP 互联网连接 (Quick UDP Internet Connections)
RDMA	远程直接内存访问 (Remote Direct Memory Access)
PFC	优先级流量控制 (Priority Flow Control)
INT	带内网络遥测 (In-band Network Telemetry)
HPCC	高精度拥塞控制 (High Precision Congestion Control)
WAN	广域网 (Wide Area Network)
Pod	交换机组 (Point of Delivery)
EMA	指数移动平均 (Exponential Moving Average)
IPC	进程间通信 (Inter-Process Communication)

第一章 绪论

1.1 研究背景与动机

随着云计算、大数据、人工智能、移动互联网和物联网技术的蓬勃发展，端到端网络传输的服务对象正在从固定主机扩展到手机、笔记本、台式机、边缘节点和传感设备等多模态终端，传输路径也从近端局域链路延伸到跨城、跨域、蜂窝接入、无线局域网和卫星通信等远距离复杂网络。传输控制协议（Transmission Control Protocol, TCP）作为互联网传输层的核心协议，承载了大量关键业务流量，其拥塞控制机制的性能直接影响网络资源利用效率、应用响应延迟和用户服务质量。网络拥塞控制是计算机网络领域最具挑战性的研究课题之一，自互联网诞生以来一直是学术界和工业界持续关注的焦点问题。

在现代端到端网络中，链路与终端条件呈现出更强的异构性。一方面，城域/广域网、蜂窝网络、WiFi 与卫星链路具有不同的 RTT 量级、丢包背景、可用带宽和链路抖动；另一方面，手机、电脑、边缘设备等终端在处理能力、无线接入方式、移动性和业务负载上差异显著。这种长距离、多接入、多终端并存的网络环境对传统拥塞控制算法提出了前所未有的严峻挑战。

现代网络应用承载着多种类型的业务负载，这些业务对网络性能有着截然不同的需求。移动视频、实时会议和云游戏需要稳定的低延迟与低抖动；分布式存储、内容分发和大文件同步需要高吞吐量以支持大规模数据传输；边缘计算、工业物联网和车联网则同时关注可靠性、实时性和终端资源约束。单一的拥塞控制策略难以同时满足这些多样化的性能需求。

更为复杂的是，远距离和多接入传输中的拥塞信号更难解释。丢包可能来自真实队列溢出，也可能来自无线误码、链路切换或终端侧调度抖动；RTT 升高既可能表示排队，也可能来自路径变化、接入链路波动或终端处理延迟；移动终端后台同步、视频传输、RPC 突发、备份任务和多租户共享链路还会造成短时间带宽挤占。传统单一信号驱动的拥塞控制方法难以在这些场景中稳定区分拥塞程度。

传统的 TCP 拥塞控制算法在这种复杂的网络环境中显露出诸多不足。基于丢包的算法（如 TCP NewReno^[1]、TCP CUBIC^[2]）将数据包丢失作为拥塞信号，但丢包是拥塞的滞后指示器，通常意味着网络缓冲区已经溢出，排队延迟已经积累到较高水平。基于延迟的算法（如 TCP Vegas^[3]、TCP BBR^[4]）尝试通过监测 RTT 变化提前感知拥塞，但 RTT 测量易受多种因素干扰，且在与基于丢包算法共存时存在公平性问题。

显式拥塞通知（Explicit Congestion Notification, ECN）机制为解决上述问题提供了新的思路。ECN 允许网络设备在队列长度超过阈值时标记数据包而非丢弃，从而实现更早、更精确的拥塞信号传递。数据中心 TCP（DCTCP^[5]）验证了 ECN 细粒度反馈在特定低时延网络中的有效性，但其适用前提较强；在远距离、多接入和多模态终端并存的环境中，现有算法对

ECN、RTT、丢包和协议栈状态的联合利用仍不充分，大多采用静态参数配置，缺乏对动态网络环境的自适应能力。

近年来，机器学习特别是强化学习技术的快速发展为网络拥塞控制带来了革命性的新思路^[6]。强化学习（Reinforcement Learning, RL）是一种通过与环境交互来学习最优策略的机器学习方法。在强化学习框架中，智能体（Agent）通过观察环境状态、执行动作并获得奖励来不断优化决策策略。将强化学习应用于网络拥塞控制，理论上能够实现：（1）自适应地学习网络环境特征，无需人工设计复杂的控制规则；（2）综合利用多种拥塞信号进行决策；（3）根据网络条件动态调整控制参数；（4）良好的可扩展性和通用性。

然而，现有基于强化学习的拥塞控制方案仍存在诸多问题：状态空间设计不完善，通常只使用 4-6 维的有限状态参数；奖励函数设计不合理，难以平衡多个优化目标；缺乏对 ECN 信号的分级利用；算法验证效率低下等。这些问题限制了强化学习在拥塞控制领域的实际应用效果。

1.2 研究目标与主要贡献

针对上述问题，本文提出了 Swift 算法——一种基于强化学习的多信号融合自适应网络拥塞控制方法。”Swift”意为”雨燕”，雨燕以其轻盈敏捷的飞行姿态和清脆悦耳的歌声著称。选用”Swift”作为算法名称，寓意本算法追求的设计目标：如同雨燕般轻盈敏捷，实现高吞吐量（High Throughput）与低延迟（Low Latency）的完美平衡；如同雨燕对环境变化的敏锐感知，实现对多种拥塞信号的快速响应和智能融合。

图1.1展示了 Swift 算法的三项核心贡献及各组件之间的逻辑关系。

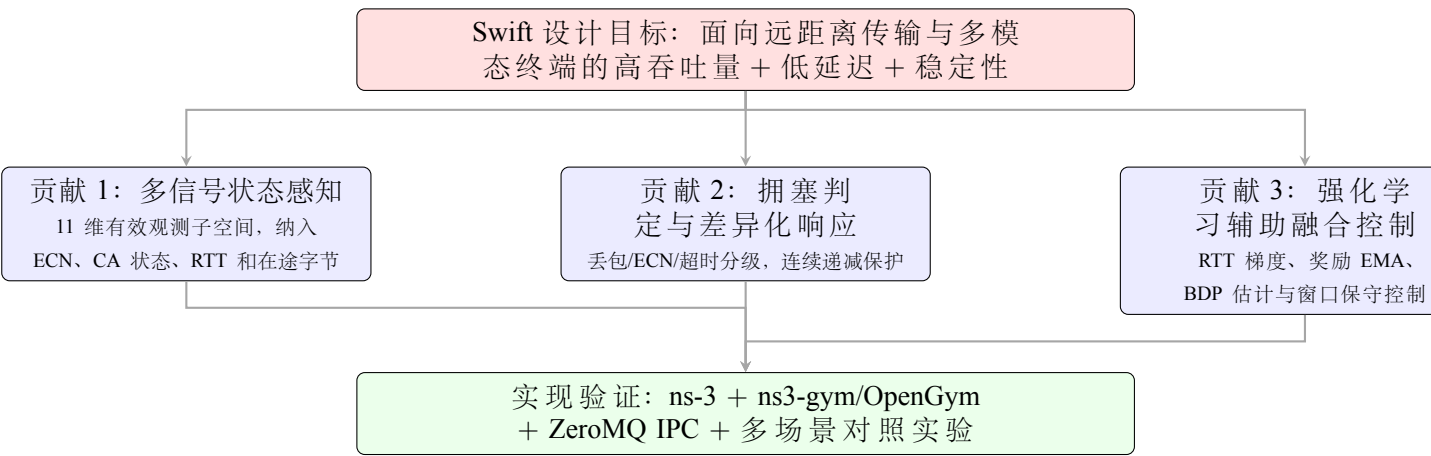


图 1.1 Swift 算法核心贡献概览

本文的主要研究贡献如下：

(1) 面向远距离与异构终端的多信号状态感知

本文设计了一个 11 维有效 RL 观测子空间（嵌入于 15 元素 OpenGym Box 传输容器，另含套接字 UUID、环境类型、仿真时间、节点 ID 四项跨进程路由用的元数据通道），将显式拥塞通知（ECN）状态、拥塞控制状态机状态、拥塞事件类型、RTT 和在途字节等 TCP 协议栈信息纳入强化学习状态表示。与现有方案通常采用的 4–6 维简化状态空间相比，Swift 的观测空间能够为手机、电脑、边缘节点和长距离链路中的 TCP 连接提供更全面、更精细的状态感知能力，为精准的拥塞控制决策奠定基础。

(2) 多信号融合拥塞判定与差异化响应

本文提出了一种多信号融合的拥塞判定方法，采用分层优先级机制综合分析丢包、超时、ECN 和拥塞控制状态信号。该方法的核心思想是：不同类型的拥塞信号具有不同的可靠性、时序特征和严重程度，应当采用差异化处理策略。具体而言，丢包和超时代表较严重的拥塞或路径异常，ECN 标记代表队列溢出前的早期预警，拥塞控制状态机则提供协议栈内部语义。基于该判定结果，Swift 针对丢包、ECN 和超时分别采用 0.70、0.75 和 0.50 的窗口保留因子，并引入连续缩减保护机制，限制最多 3 次连续缩减后保持窗口不变，避免因连续拥塞事件导致窗口过度收缩。

(3) 强化学习辅助的自适应融合控制

本文实现了强化学习辅助的自适应融合控制策略，根据 RTT 膨胀比、奖励信号指数移动平均和连续增长趋势动态调整乘性增加因子 α ，并将其限制在 $[0.85, 1.30]$ 范围内以保证控制稳定性。在窗口计算中，Swift 采用窗口化最大值过滤方法估计 BDP，并围绕 $\alpha \times BDP$ 目标窗口进行有界步长的渐进拉升或回落；同时结合 RTT 约束下的管道利用率感知加性递增机制，在吞吐量、延迟和稳定性之间实现自适应折中。上述策略在 ns-3 与 ns3-gym/OpenGym 框架中实现，并通过 ZeroMQ 进程间通信和多场景对照实验完成验证。

1.3 论文组织结构

本文的其余部分组织如下：

第 2 章介绍相关工作，包括传统拥塞控制算法、基于机器学习的拥塞控制方法、显式拥塞通知机制、强化学习在网络领域的应用等。

第 3 章详细描述 Swift 算法的设计与实现，包括系统架构、观测空间设计、多信号融合拥塞检测、差异化拥塞响应、自适应参数调优、融合控制策略等核心组件。

第 4 章介绍实验评估，包括实验环境配置、评测指标、对比算法、实验场景设计以及详细的实验结果分析。

第 5 章讨论 Swift 算法的适用边界和未来研究方向。

第 6 章总结全文并展望未来工作。

第二章 相关工作

网络拥塞控制是计算机网络领域的核心研究课题之一，自互联网诞生以来一直是学术界和工业界关注的焦点。本章从传统拥塞控制算法、显式拥塞通知机制、基于机器学习的拥塞控制方法三个方面综述相关研究工作，并分析各方法的优缺点及其对 **Swift** 算法设计的启示。

图2.1展示了拥塞控制算法的分类体系及其演进关系。

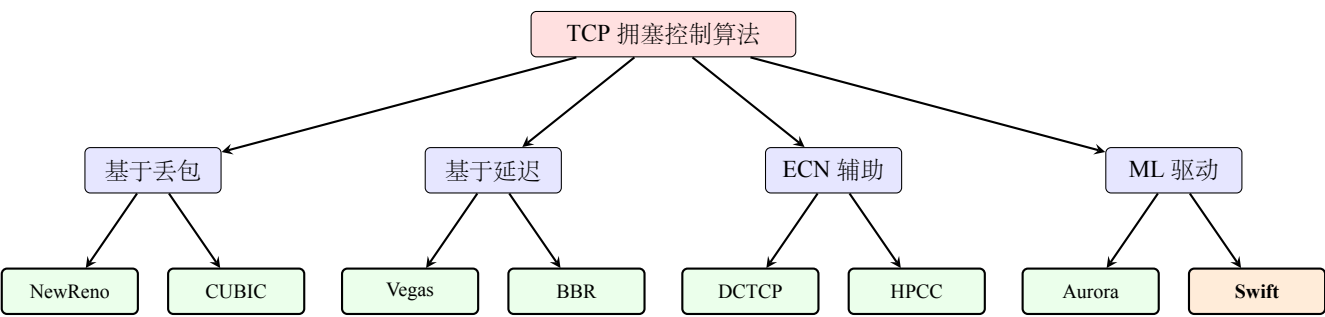


图 2.1 拥塞控制算法分类体系

2.1 传统拥塞控制算法

传统的 TCP 拥塞控制算法根据其使用的拥塞信号类型，可以分为基于丢包的算法、基于延迟的算法和混合算法三大类。

2.1.1 基于丢包的拥塞控制算法

基于丢包的拥塞控制算法将数据包丢失作为网络拥塞的主要判断信号。这类算法的核心思想是：当发送端检测到数据包丢失（通过超时或重复确认判断）时，认为网络发生了拥塞，需要降低发送速率以缓解拥塞。

TCP Tahoe 和 **TCP Reno** 是最早的拥塞控制算法，由 Van Jacobson 于 1988 年提出^[7]。TCP Tahoe 引入了慢启动(Slow Start)、拥塞避免(Congestion Avoidance)和快速重传(Fast Retransmit)三个核心机制。慢启动机制在连接建立初期指数级增加拥塞窗口，快速探测可用带宽；拥塞避免机制在窗口达到阈值后线性增加窗口，谨慎探测额外带宽；快速重传机制在收到三个重复确认时立即重传丢失的数据包，而无需等待超时。TCP Reno 在 Tahoe 的基础上增加了快速恢复(Fast Recovery)机制，在快速重传后将窗口减半而非重置为 1，提高了丢包后的恢复效率。

TCP NewReno^[1] 是对 TCP Reno 的改进，主要解决了 Reno 在多个数据包丢失时性能下降的问题。NewReno 修改了快速恢复机制的行为：在快速恢复期间，如果收到部分确认(Partial

ACK，确认了部分但非全部已发送数据），则继续保持在快速恢复状态，直到所有丢失的数据包都被成功重传。这种改进使得 NewReno 能够更高效地处理突发丢包情况，成为 RFC 6582 中定义的标准 TCP 拥塞控制算法之一。

TCP CUBIC^[2] 是目前 Linux 操作系统的默认拥塞控制算法，自 Linux 2.6.19 版本开始采用。CUBIC 的核心创新是采用三次函数（Cubic Function）模型调整拥塞窗口大小：

$$W(t) = C \cdot (t - K)^3 + W_{max} \quad (2.1)$$

其中， $W(t)$ 是时刻 t 的窗口大小， C 是缩放因子， $K = \sqrt[3]{\frac{W_{max} \cdot \beta}{C}}$ 是从上次拥塞事件恢复到 W_{max} 所需的时间， W_{max} 是上次拥塞事件前的窗口大小， β 是窗口缩减因子。

CUBIC 的三次函数模型具有以下特点：（1）在远离 W_{max} 时快速增长，在接近 W_{max} 时缓慢增长，实现了对可用带宽的稳定探测；（2）窗口增长速度与 RTT 无关，避免了传统 AIMD 算法在高 RTT 链路上收敛缓慢的问题；（3）在 W_{max} 附近有一个“平台期”，减少了在最优工作点附近的窗口振荡。

TCP BIC^[8] 是 CUBIC 的前身，采用二分搜索（Binary Search）算法调整拥塞窗口。BIC 的基本思想是：上次拥塞发生时的窗口 W_{max} 是带宽上限的估计，当前窗口是带宽下限的估计，通过二分搜索在两者之间寻找最优窗口。BIC 算法的窗口增长函数如下：

$$W_{new} = W_{current} + \frac{W_{max} - W_{current}}{2} \quad (2.2)$$

当窗口接近 W_{max} 时，BIC 采用加性增加策略以避免过冲。BIC 算法在高带宽延迟积网络中表现良好，但其窗口增长行为对 RTT 不敏感，在与其他 TCP 变体共存时可能产生公平性问题。

基于丢包的算法存在以下共同缺陷：

（1）**拥塞信号滞后**：丢包是网络拥塞的滞后指示器。网络设备的缓冲区在溢出丢包前已经积累了大量排队数据，导致排队延迟显著增加。在深缓冲区（Deep Buffer）网络环境中，这种延迟可能达到数十甚至数百毫秒级别，严重影响延迟敏感型应用的性能。

（2）**缓冲区膨胀**：基于丢包算法的典型行为是持续增加发送速率直到发生丢包，这会导致网络缓冲区长期处于高占用状态，产生“Bufferbloat”（缓冲区膨胀）问题^[9]。缓冲区膨胀不仅增加了网络延迟，还降低了网络的整体吞吐量，因为大量带宽被用于传输最终会被丢弃的数据包。

（3）**无法区分拥塞程度**：基于丢包算法只能检测“有丢包”或“无丢包”两种状态，无法区分轻度拥塞和严重拥塞。因此，无论拥塞程度如何，算法都采用统一的窗口缩减策略，在轻度拥塞时可能响应过度，在严重拥塞时又可能响应不足。

(4) **高 BDP 网络收敛缓慢**: 在高带宽延迟积网络中, 基于丢包算法的收敛速度极为缓慢。以 10Gbps 带宽、100 微秒 RTT 的链路为例, BDP 约为 125KB, 需要约 86 个 MSS 的窗口才能充分利用带宽。从初始窗口增长到 86 个 MSS 需要较长时间, 丢包后的恢复更是如此。

2.1.2 基于延迟的拥塞控制算法

基于延迟的拥塞控制算法通过监测往返时延 (RTT) 的变化来推断网络拥塞状态, 试图在丢包发生前就感知到拥塞并采取措施。这类算法的核心假设是: 网络拥塞会导致队列积累, 进而导致 RTT 增加。

TCP Vegas^[3] 是最早的基于延迟拥塞控制算法之一, 由 Lawrence Brakmo 和 Larry Peterson 于 1994 年提出。Vegas 通过比较期望吞吐量和实际吞吐量来估计网络中的排队数据量。期望吞吐量定义为:

$$Expected = \frac{cwnd}{BaseRTT} \quad (2.3)$$

实际吞吐量定义为:

$$Actual = \frac{cwnd}{RTT} \quad (2.4)$$

其中, $BaseRTT$ 是测量到的最小 RTT, 被认为是无排队情况下的传播延迟。差值 $\Delta = Expected - Actual$ 表示网络中排队的的数据量。Vegas 根据 Δ 的大小调整拥塞窗口:

$$cwnd = \begin{cases} cwnd + 1 & \text{如果 } \Delta < \alpha \\ cwnd - 1 & \text{如果 } \Delta > \beta \\ cwnd & \text{否则} \end{cases} \quad (2.5)$$

其中, α 和 β 是预设的阈值参数, 通常设置为 $\alpha = 1$ 、 $\beta = 3$ 。

TCP BBR^[4] (Bottleneck Bandwidth and Round-trip propagation time) 是 Google 于 2016 年提出的基于延迟算法, 是近年来最具影响力的拥塞控制算法之一。BBR 的核心思想是通过显式估计瓶颈带宽 (Bottleneck Bandwidth, BtlBw) 和最小 RTT (RTT_{prop}), 以带宽延迟积为目标调整发送速率。

BBR 通过维护一个滑动窗口来跟踪最大投递率 (作为 BtlBw 的估计) 和最小 RTT (作为 RTT_{prop} 的估计)。发送速率和拥塞窗口按以下方式计算:

$$pacing_rate = \frac{pacing_gain \times BtlBw}{6} \quad (2.6)$$

$$cwnd = cwnd_gain \times BDP = cwnd_gain \times BtlBw \times RTT_{prop} \quad (2.7)$$

BBR 采用周期性的增益调整策略，在不同阶段采用不同的 $pacing_gain$ 和 $cwnd_gain$ 值，以实现带宽探测和 RTT 测量。

TCP Copa^[10] 是 MIT 于 2018 年提出的基于延迟算法，采用了与 Vegas 类似但更精细的 RTT 监测机制。Copa 将目标排队延迟设定为 $RTT_{standing} - RTT_{min}$ 的函数，其中 $RTT_{standing}$ 是最近一个 RTT 窗口内的最小 RTT。Copa 采用以下窗口调整规则：

$$cwnd = cwnd + \frac{1}{cwnd \cdot \delta} \quad \text{或} \quad cwnd = cwnd - \frac{1}{cwnd \cdot \delta} \quad (2.8)$$

其中， δ 是目标延迟参数，默认值为 0.5。Copa 还引入了“竞争模式”检测机制，当检测到与基于丢包算法的流竞争时，会切换到更激进的发送策略。

基于延迟的算法存在以下共同问题：

(1) **RTT 测量不稳定**：RTT 测量受多种因素影响，包括操作系统调度延迟、网络路径抖动、网络拓扑变化、对端主机处理延迟等。在高速网络中，这些干扰因素可能与实际排队延迟处于同一数量级，导致基于 RTT 的拥塞推断不可靠。

(2) **公平性问题**：基于延迟算法在与基于丢包算法共存时存在严重的公平性问题。由于基于延迟算法对拥塞信号更敏感，会在 RTT 增加时提前降低发送速率，而基于丢包算法继续增加窗口直到丢包。研究表明，在与 CUBIC 流共存时，BBR 流的吞吐量可能下降 30% 以上^[11]。

(3) **参数敏感性**：基于延迟算法的性能对参数设置较为敏感。例如，Vegas 的 α 和 β 参数、Copa 的 δ 参数都需要根据具体网络环境进行调优，而静态参数难以适应动态变化的网络条件。

2.1.3 混合拥塞控制算法

混合拥塞控制算法综合利用丢包和延迟两种信号进行拥塞控制，试图结合两类算法的优点。

Compound TCP^[12] 是 Microsoft 开发的混合算法，在 Windows Vista 及后续版本中使用。Compound TCP 在传统的基于丢包的窗口增长之上叠加了一个基于延迟的“快速”窗口分量，在网络空闲时快速增长，在队列积累时减速增长。

TCP Illinois^[13] 根据 RTT 变化调整 AIMD 参数，在低延迟时采用更激进的增加策略，在高延迟时采用更保守的增加策略和更激进的减少策略。

TCP Westwood^[14] 通过测量 ACK 到达率来估计可用带宽，在丢包时根据估计的带宽而非固定比例调整慢启动阈值。

混合算法在一定程度上缓解了单一信号类型的局限性，但也增加了算法复杂性，且参数调优更加困难。

2.2 显式拥塞通知机制

显式拥塞通知（Explicit Congestion Notification, ECN）是由 IETF 在 RFC 3168^[15] 中定义的一种网络层辅助拥塞控制机制。ECN 通过修改 IP 报头中的服务类型字段（Type of Service, ToS）来传递拥塞信息，实现了比隐式丢包信号更早、更精确的拥塞通知。

2.2.1 ECN 工作原理

ECN 在 IP 报头中使用两个比特（ECN 字段），定义了四种码点：

- 00: Non-ECT（非 ECN 能力传输），表示端点不支持 ECN
- 01: ECT(1)（ECN 能力传输 1），表示端点支持 ECN
- 10: ECT(0)（ECN 能力传输 0），表示端点支持 ECN
- 11: CE（Congestion Experienced，拥塞经历），由路由器标记，表示经历了拥塞

ECN 的工作流程如下：发送端在发送数据包时将 ECN 字段设置为 ECT(0) 或 ECT(1)，表明该数据包支持 ECN。当网络设备（如路由器）检测到队列长度超过预设阈值时，将 ECN 字段从 ECT(0/1) 修改为 CE，而不是丢弃数据包。接收端收到 CE 标记的数据包后，在 TCP 报头中设置 ECE（ECN-Echo）标志，将拥塞信息反馈给发送端。发送端收到 ECE 标志后，采取拥塞响应措施（如降低发送速率），并在随后的数据包中设置 CWR（Congestion Window Reduced）标志，通知接收端已经响应了拥塞通知。图2.2展示了 ECN 信号传递的完整工作流程。

2.2.2 ECN 相关拥塞控制算法

DCTCP^[5]（Data Center TCP）是 Microsoft 和 Stanford 于 2010 年提出的面向数据中心环境的拥塞控制算法，是 ECN 应用的典范。DCTCP 的核心创新是利用 ECN 标记的**比例**（而非仅仅是有无）来进行细粒度的窗口调整。DCTCP 维护一个指数加权移动平均（EWMA）的 ECN 标记比例 α ：

$$\alpha = (1 - g) \cdot \alpha + g \cdot F \quad (2.9)$$

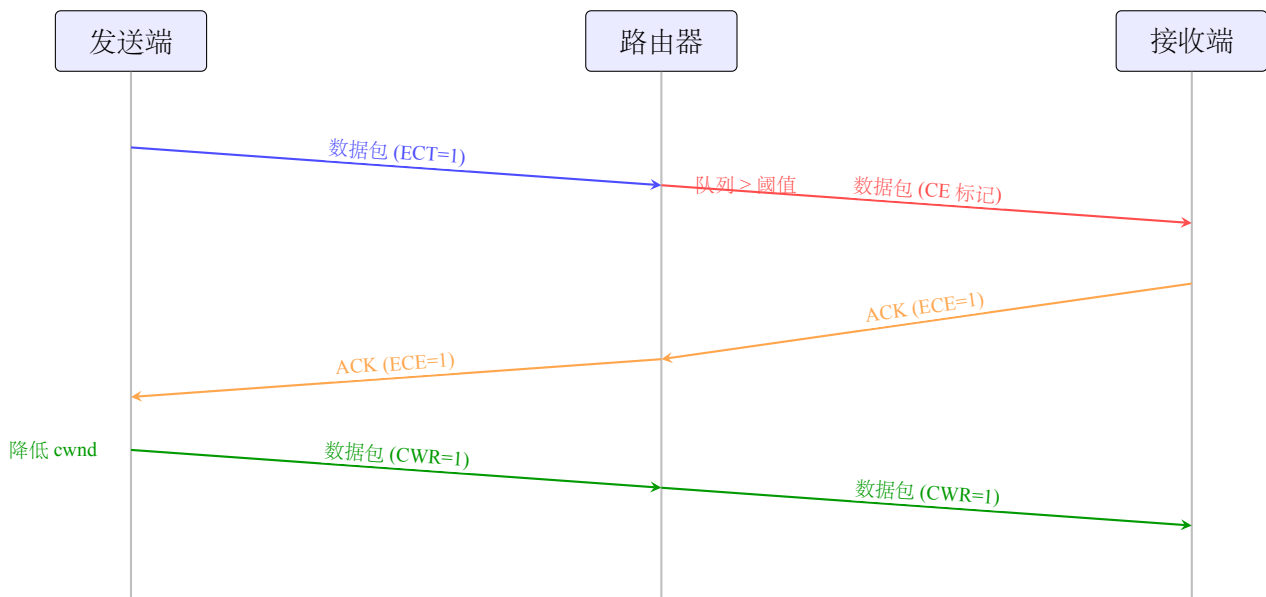


图 2.2 ECN 信号传递工作流程

其中, g 是平滑因子 (通常为 $1/16$), F 是最近一个 RTT 内被标记的数据包比例。拥塞窗口根据 α 进行调整:

$$cwnd = cwnd \times \left(1 - \frac{\alpha}{2}\right) \quad (2.10)$$

这种设计使得 DCTCP 能够根据拥塞程度进行比例响应: 轻度拥塞时 (α 较小) 轻微降低窗口, 严重拥塞时 (α 较大) 大幅降低窗口。

DCTCP 要求网络交换机配置适当的 ECN 标记阈值, 通常使用 RED (Random Early Detection) 或其变体。推荐的配置是将标记阈值设置为较低的值 (如 20 个数据包或 BDP 的 10%), 以实现早期拥塞通知。

ECN++^[16] 是对 DCTCP 的改进, 引入了更精细的 ECN 标记策略和响应机制。ECN++ 在接收端累积 ECN 标记计数, 而非仅使用二元 ECE 标志, 从而传递更丰富的拥塞信息。

HPCC^[17] (High Precision Congestion Control) 是阿里巴巴于 2019 年提出的高精度拥塞控制算法, 利用网络设备提供的精确拥塞信息 (如队列长度、带宽利用率) 进行拥塞控制。HPCC 要求网络设备支持 INT (In-band Network Telemetry) 功能。

ECN 机制的主要优势在于: (1) 能够在丢包发生前提供明确的拥塞信号; (2) 避免了因丢包导致的重传开销; (3) 信号来自网络设备的显式标记, 比推测性的 RTT 测量更可靠。然而, ECN 的有效使用需要网络设备的支持和正确配置, 且现有算法对 ECN 信号的利用还不够充分。

2.3 基于机器学习的拥塞控制方法

近年来，机器学习技术在网络拥塞控制领域的应用受到了广泛关注。根据所采用的学习范式，可以分为监督学习方法、强化学习方法和其他方法。

2.3.1 监督学习方法

监督学习方法通过从标记数据中学习拥塞控制策略。

Remy^[18] 是 MIT 于 2013 年提出的开创性工作，采用离线优化方法生成拥塞控制规则。Remy 将拥塞控制问题形式化为一个优化问题，在给定网络模型的情况下，搜索最优的窗口调整规则。Remy 的规则形式为：在特定状态下（由 RTT 测量、ACK 间隔等特征定义），执行特定的窗口增量和发送间隔调整。

Remy 的优点是能够针对特定网络环境生成最优规则，在目标场景下表现优异。其局限性在于：（1）需要准确的网络模型，对模型误差敏感；（2）离线生成的规则难以适应动态变化的网络环境；（3）状态空间划分的粒度影响算法性能和复杂度。

2.3.2 强化学习方法

强化学习方法通过与网络环境的交互来学习最优的拥塞控制策略，能够适应动态变化的网络条件。

Aurora^[19] 是 CMU 于 2019 年提出的基于深度强化学习的拥塞控制算法。Aurora 采用 PPO (Proximal Policy Optimization) 算法训练神经网络策略，输入网络状态特征（延迟梯度、延迟比率、发送比率等），输出发送速率调整因子。Aurora 的状态空间包含 10 个特征，涵盖延迟、吞吐量和丢包率的历史统计信息。

Aurora 的奖励函数设计为吞吐量和延迟的加权组合：

$$reward = throughput - \delta \cdot delay - \lambda \cdot loss \quad (2.11)$$

其中， δ 和 λ 是权重参数，控制延迟和丢包的惩罚程度。

Orca^[20] 是 UC Berkeley 于 2020 年提出的强化学习拥塞控制算法，采用经典的 AIMD 框架，但使用强化学习来动态调整 AIMD 参数。Orca 的核心思想是：AI 参数（增加因子 α 和减少因子 β ）不应是固定值，而应根据网络状态动态调整。

PCC^[21] (Performance-oriented Congestion Control) 是 UIUC 提出的基于在线学习的拥塞控制算法。PCC 采用实用函数 (Utility Function) 指导发送速率调整，通过在线实验探索最优速率。

$$U(x) = T \cdot x^\alpha - \delta \cdot L \cdot x - \beta \cdot D \cdot x \quad (2.12)$$

其中, x 是发送速率, T 是吞吐量, L 是丢包率, D 是延迟, α 、 δ 、 β 是参数。PCC 通过梯度上升方法最大化实用函数。

现有基于强化学习的方法存在以下共同问题:

- (1) **状态空间设计不完善**: 现有方案通常只使用 4-6 维的简化状态参数, 未能充分利用 TCP 协议栈提供的丰富状态信息, 特别是 ECN 相关状态和拥塞控制事件信息。
- (2) **奖励函数设计困难**: 吞吐量、延迟、公平性、丢包率等多个优化目标之间存在权衡关系, 设计能够平衡多目标的奖励函数是一个挑战。
- (3) **训练效率低下**: 网络仿真的计算开销较大, 限制了强化学习算法的训练效率。
- (4) **泛化能力有限**: 在特定网络环境下训练的策略可能难以泛化到其他环境。

2.4 远距离与异构终端网络特性分析

为了更好地理解 Swift 算法的设计动机, 本节分析远距离传输、多接入链路与异构终端并存环境中的网络特性。此类环境通常同时包含手机、笔记本电脑、台式机、边缘节点和云端服务等端系统, 端到端路径可能跨越无线接入、城域网、广域网和卫星链路, 网络条件随终端能力、接入方式和路径拥塞状态动态变化。

2.4.1 路径与接入特性

远距离与异构终端传输场景具有以下特点:

- (1) **往返时延跨度大**: 不同路径的 RTT 可能从毫秒级扩展到数百毫秒级。较大的带宽时延积使发送端需要更准确地估计可容纳的在途数据量, 否则容易出现带宽利用不足或队列持续驻留。
- (2) **接入链路异构**: 终端可能通过有线以太网、Wi-Fi、蜂窝网络或卫星链路接入网络。不同接入链路在带宽、误码、调度粒度和队列管理方式上差异显著, 使固定参数的拥塞控制难以同时适配多类路径。
- (3) **终端能力差异明显**: 手机、笔记本电脑、桌面终端和边缘设备在处理能力、网卡能力、缓冲区规模和操作系统协议栈配置方面存在差异, 这会影响 ACK 反馈节奏、突发发送行为和拥塞窗口恢复速度。
- (4) **路径状态动态变化**: 无线信道波动、移动性、跨流量突发和路由变化会导致 RTT、吞吐量和丢包风险快速变化。拥塞控制算法需要利用多维状态信号进行及时感知, 而不能仅依赖单一丢包反馈。

2.4.2 流量特性分析

远距离与多终端网络中的流量负载具有明显的多样性：

(1) **业务类型多样**：文件同步、视频会议、网页访问、远程桌面、云游戏、在线备份和模型数据传输等业务可能同时存在，对吞吐量、时延和稳定性的要求不同。

(2) **突发跨流量普遍**：非响应式 UDP 流、周期性上报、后台同步和多媒体码流可能在短时间内占用瓶颈队列，放大 RTT 抖动并干扰 TCP 长流的窗口控制。

(3) **长短流混合**：短流关注完成时间，长流关注持续吞吐量和稳定性。若拥塞控制响应过慢，短流可能受到排队延迟影响；若响应过激，长流又可能出现带宽利用不足。

(4) **反馈信号不完全一致**：丢包、ECN、RTT 膨胀和协议栈状态分别反映不同层面的网络变化。多信号融合能够降低单一信号误判带来的风险。

2.4.3 应用延迟需求分析

不同应用对网络延迟和吞吐量的需求并不相同：

(1) **实时交互业务**：视频会议、云游戏和远程协作要求持续低延迟，拥塞控制需要避免队列长期驻留。

(2) **大文件与数据同步业务**：云盘同步、备份和数据分发更关注带宽利用率，需要在不引发持续拥塞的前提下充分填充高 BDP 路径。

(3) **移动终端业务**：移动设备受无线链路波动和终端能耗约束影响，更需要平滑、稳定且可自适应的窗口调节策略。

(4) **边缘与云服务访问**：边缘计算、远程推理和跨区域服务调用同时关注时延、吞吐量和鲁棒性，需要算法在多目标之间动态权衡。

Swift 算法的“高吞吐、低排队、强鲁棒”设计目标正是为了适应上述复杂场景：在路径条件良好时保持较高带宽利用率，在拥塞或突发跨流量出现时及时降低排队和丢包风险。

2.4.4 网络设备与反馈机制特性

网络设备和协议栈反馈机制对拥塞控制算法有重要影响：

(1) **缓冲区规模差异**：不同接入设备和中间节点的缓冲区规模差异较大，浅缓冲链路更容易丢包，深缓冲链路更容易产生排队时延。

(2) **ECN 支持程度不一**：部分网络设备支持 ECN 标记，可以在队列溢出前提供早期拥塞信号。算法需要在可获得 ECN 信号时充分利用该信号，同时也在无 ECN 场景下仍保持基于丢包和 RTT 的稳健响应。

(3) **终端协议栈可观测性增强**：现代协议栈能够暴露拥塞状态、拥塞事件、RTT 样本和在途字节等信息，为强化学习辅助的多维状态建模提供基础。

(4) **部署形态多样**：拥塞控制逻辑既可以在内核 TCP 栈中实现，也可以在用户态传输协议或仿真环境中验证。统一的状态感知和窗口控制框架有助于后续迁移与工程化。

2.5 拥塞控制的理论基础

本节介绍网络拥塞控制的理论基础，包括流量模型、排队论分析和控制论视角，为理解 Swift 算法的设计决策提供理论支撑。

2.5.1 网络流量模型

网络流量建模是拥塞控制研究的基础。常用的流量模型包括：

(1) **泊松模型**：传统上假设数据包到达服从泊松过程，即到达间隔服从指数分布。泊松模型的数学性质简单，便于分析，但实际网络流量通常表现出更强的突发性。

(2) **自相似模型**：研究表明，网络流量具有自相似（Self-Similar）特性，即在不同时间尺度上呈现相似的统计特性。自相似流量可以用分数布朗运动（Fractional Brownian Motion）或 Pareto 分布建模。设流量过程为 $X(t)$ ，其自相似性定义为：

$$X(t) \stackrel{d}{=} m^{-H} X(mt), \quad \forall m > 0 \quad (2.13)$$

其中 H 是 Hurst 参数 ($0.5 < H < 1$)， $\stackrel{d}{=}$ 表示同分布。

(3) **ON-OFF 模型**：TCP 流量可以用 ON-OFF 模型描述，其中 ON 状态对应数据传输，OFF 状态对应等待。如果 ON 和 OFF 的持续时间服从重尾分布，聚合流量将表现出自相似特性。

(4) **流体模型**：在分析大规模网络时，可以将离散的数据包流近似为连续的流体流。设发送速率为 $x(t)$ ，队列长度为 $q(t)$ ，链路容量为 C ，则流体模型为：

$$\dot{q}(t) = x(t) - C, \quad \text{当 } q(t) > 0 \text{ 或 } x(t) > C \quad (2.14)$$

Swift 算法的设计考虑了流量的突发性特征。当前实现优先使用显式丢包回调和 ECN 状态作为拥塞判定信号，并将 RTT 膨胀用于 α 参数自适应调节，从而在突发跨流量和路径 RTT 波动下避免仅依赖单一信号作出窗口决策。

2.5.2 排队论分析

排队论为分析网络缓冲区行为提供了理论工具。考虑单个瓶颈链路，可以建模为 M/M/1 队列（泊松到达、指数服务时间、单服务器）或更一般的 G/G/1 队列。

M/M/1 队列的稳态性质：设到达率为 λ ，服务率为 μ ，负载为 $\rho = \lambda/\mu$ （ $\rho < 1$ 时队列稳定），则：

平均队列长度：

$$\bar{Q} = \frac{\rho}{1 - \rho} \quad (2.15)$$

平均等待时间（Little 公式）：

$$\bar{W} = \frac{\bar{Q}}{\lambda} = \frac{1}{\mu - \lambda} \quad (2.16)$$

队列长度分布：

$$P(Q = k) = (1 - \rho)\rho^k, \quad k = 0, 1, 2, \dots \quad (2.17)$$

从排队论角度，Swift 的低延迟特性源于其维持较低的平均负载 ρ 。通过对 ECN 信号的敏感响应，Swift 在 ρ 接近 1 之前就降低发送速率，从而保持 $\bar{Q}$ 和 $\bar{W}$ 在较低水平。

ECN 标记概率：设队列长度为 Q ，ECN 标记阈值为 K ，则在 RED (Random Early Detection) 策略下：

$$P(\text{标记}|Q) = \begin{cases} 0 & Q < \min_th \\ \frac{Q - \min_th}{\max_th - \min_th} & \min_th \leq Q < \max_th \\ 1 & Q \geq \max_th \end{cases} \quad (2.18)$$

Swift 对 ECN 的利用采用直接状态检测机制：当 `ecnState` 为 `ECN_CE_RCVD` 或 `ECN_ECE_RCVD` 时，判定存在 ECN 拥塞信号，触发保留因子为 0.75 的早期预警窗口缩减。这种直接检测方式避免了基于事件率的间接测量，具有更低的响应延迟。

2.5.3 控制论视角

从控制论角度，TCP 拥塞控制可以建模为反馈控制系统。系统框图包括：

- 被控对象：网络（包括链路、路由器、队列）
- 控制器：拥塞控制算法
- 反馈信号：ACK、RTT、ECN 标记、丢包
- 控制输入：发送窗口或发送速率

- 干扰：跨流量、背景噪声

线性化分析：在稳态点附近，TCP 系统可以线性化。设稳态发送速率为 x^* ，稳态队列长度为 q^* ，令 $\delta x = x - x^*$ ， $\delta q = q - q^*$ ，则线性化系统为：

$$\dot{\delta q} = \delta x \quad (2.19)$$

$$\delta x(t) = -K \cdot \delta q(t - \tau) \quad (2.20)$$

其中 K 是控制增益， τ 是反馈延迟（约等于 RTT）。

系统的特征方程为：

$$s + Ke^{-s\tau} = 0 \quad (2.21)$$

稳定性条件（小增益定理）：

$$K\tau < \frac{\pi}{2} \quad (2.22)$$

这意味着在高延迟网络中，控制增益 K 必须足够小以保证稳定性，但过小的 K 会导致收敛缓慢。Swift 的自适应参数调优机制可以理解为根据 τ （通过 RTT 测量）动态调整 K （通过 α 参数）。

时滞系统分析：网络的反馈延迟（RTT）使得拥塞控制成为时滞系统。时滞系统的稳定性分析比常微分方程系统更复杂。Swift 通过综合使用多种拥塞信号，减少了对单一信号（如 RTT）的依赖，提高了系统的鲁棒性。

2.6 强化学习在网络中的应用综述

强化学习技术在网络领域的应用日益广泛。本节综述强化学习在路由优化、资源分配、安全防护等方面的研究进展，为 Swift 算法的强化学习设计提供参考。

2.6.1 路由优化

强化学习可以用于学习最优路由策略。Q-routing 算法将每个路由器建模为智能体，学习到各目的地的最优下一跳。Deep Q-Network (DQN) 被应用于软件定义网络 (SDN) 中的流量工程，学习全局最优的路由配置。

与路由优化相比，拥塞控制的特点是：（1）时间尺度更细（毫秒级 vs. 秒级）；（2）状态空间更大（每个流独立状态）；（3）实时性要求更高。这些特点决定了 Swift 采用基于规则的决策机制，而非深度神经网络。

2.6.2 资源分配

在云计算和边缘计算中，强化学习被用于动态资源分配。例如，使用深度强化学习进行虚拟机放置、容器调度、带宽分配等。这些应用的决策周期通常在秒级到分钟级，对实时性要求较低。

Swift 算法借鉴了资源分配中的多目标优化思想，通过差异化响应机制平衡吞吐量和延迟两个目标。

2.6.3 安全防护

强化学习在网络安全领域也有应用，如入侵检测、DDoS 防护等。与安全应用不同，拥塞控制需要处理合法流量，不能简单地丢弃可疑流量，需要更精细的控制策略。

2.6.4 与 Swift 的关系

Swift 算法继承了强化学习的核心思想——通过与环境交互学习最优策略，但在具体实现上做了以下调整：

- 采用基于规则的策略而非神经网络策略，保证实时性和可解释性
- 将观测空间扩展到 11 维，充分利用 TCP 协议栈信息
- 实现每流独立状态管理，支持大规模并发场景
- 通过自适应参数调优实现策略的在线优化

表2.1从多个维度对比了上述代表性拥塞控制算法与 Swift 的技术特性。

表 2.1 代表性拥塞控制算法综合对比

特性	NewReno	CUBIC	BBR	DCTCP	Swift
拥塞信号类型	丢包	丢包	RTT	ECN 比例	多信号融合
ECN 利用	否	否	否	是	是
差异化响应	否	否	否	部分	是
参数自适应	否	否	否	否	是
BDP 估计	否	否	是	否	是
每流状态管理	是	是	是	是	是
可解释性	高	高	中	高	高
学习能力	无	无	无	无	在线自适应

2.7 本章小结

本章综述了网络拥塞控制领域的相关研究工作。传统的基于丢包算法（如 NewReno、CU-BIC）虽然成熟稳定，但存在拥塞信号滞后、缓冲区膨胀等问题；基于延迟算法（如 Vegas、BBR）能够提前感知拥塞，但 RTT 测量不稳定且公平性较差；ECN 机制提供了更精确的拥塞信号，但现有算法的利用还不够充分；基于机器学习的方法展现出自适应学习的潜力，但仍存在状态空间设计、奖励函数设计、训练效率等方面的问题。

Swift 算法在现有工作的基础上，通过设计完整的 11 维观测空间、提出多信号融合拥塞检测方法、实现差异化拥塞响应机制和自适应参数调优，试图解决上述问题，实现高吞吐量和低延迟的拥塞控制目标。

第三章 Swift 算法设计与实现

本章详细介绍 Swift 算法的设计与实现，包括系统架构概述、强化学习问题建模、11 维观测空间设计、多信号融合拥塞检测算法、差异化拥塞响应机制、强化学习驱动的自适应参数调优、融合控制策略以及每流独立状态管理机制。

3.1 系统架构概述

Swift 算法的系统架构如图3.1所示，由四个核心模块组成：网络仿真模块、拥塞控制环境模块、智能决策模块和进程间通信模块。

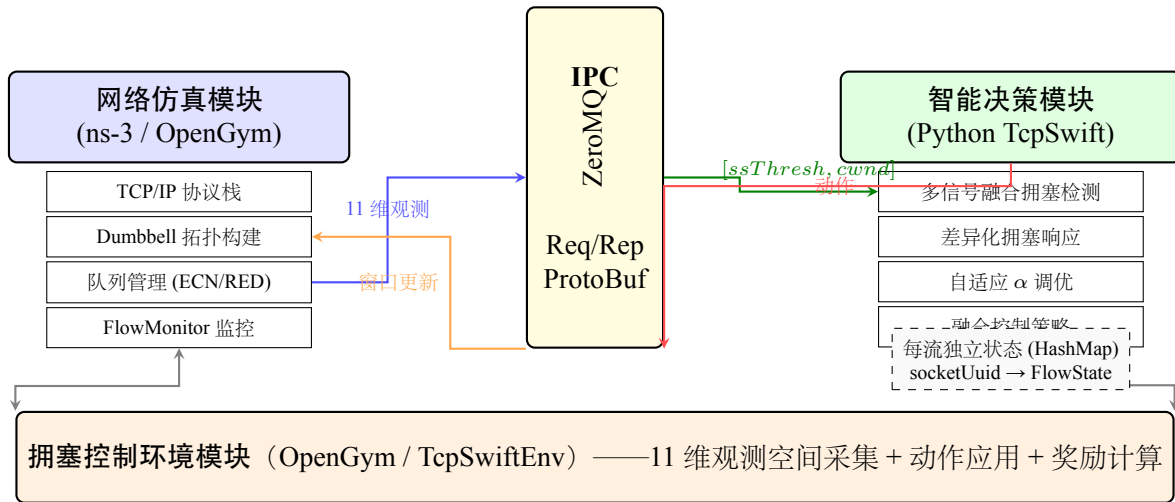


图 3.1 Swift 系统架构

网络仿真模块基于 ns-3 (Network Simulator 3) 离散事件仿真平台构建, 实现完整的 TCP/IP 协议栈。ns-3 是目前学术界和工业界广泛使用的网络仿真工具, 提供了丰富的网络协议模型和灵活的可扩展架构。网络仿真模块负责构建哑铃型等可控拓扑、模拟数据包传输过程、执行离散事件驱动的仿真调度, 并通过 FlowMonitor 等工具记录吞吐量、时延、丢包和公平性指标。

本研究中的 Swift 实验采用 ns3-gym 提供的 OpenGym 接口与 Python 智能决策模块交互。仿真进程在 TCP 回调处同步发送观测向量, Python 侧根据观测向量返回慢启动阈值和拥塞窗口动作。由于 OpenGym/ZeroMQ 采用同步请求-应答交互路径, 本文实现未将该路径与 MTP 多线程并行仿真机制结合使用。

拥塞控制环境模块基于 OpenGym 框架实现, 负责在 TCP 协议栈与强化学习智能体之间建立桥梁。OpenGym 是 ns3-gym 项目提供的强化学习环境接口, 遵循 OpenAI Gym 的标准 API 规范。拥塞控制环境模块在 TCP 协议栈的关键回调点采集状态参数, 构建 11 维观测空

间，并将智能体的决策动作应用于 TCP 连接。

拥塞控制环境模块实现了以下回调接口：

- `GetSsThresh()`：慢启动阈值获取回调，在检测到丢包时调用
- `IncreaseWindow()`：窗口增加回调，在收到正常确认时调用
- `PktsAcked()`：数据包确认回调，用于 RTT 测量和吞吐量计算
- `CongestionStateSet()`：拥塞状态设置回调，用于跟踪拥塞控制状态机变化
- `CwndEvent()`：拥塞窗口事件回调，用于捕获 ECN 标记等事件

智能决策模块采用 Python 实现，负责执行 Swift 算法的核心逻辑：多信号融合拥塞检测、差异化拥塞响应、自适应参数调优和融合控制策略计算。智能决策模块维护每个 TCP 连接的独立状态，包括历史度量缓冲区、ECN 事件跟踪数据、性能指标数据和自适应参数数据。

进程间通信模块采用 ZeroMQ (ZMQ) 消息队列实现仿真进程和决策进程之间的高效通信。通信协议采用请求-应答 (Request-Reply) 模式：仿真进程在每个回调点发送观测数据请求，决策进程返回控制动作应答。消息格式采用 Protocol Buffers 序列化，保证了通信效率和跨语言兼容性。

3.2 强化学习问题建模

Swift 算法将网络拥塞控制问题建模为马尔可夫决策过程 (Markov Decision Process, MDP)。MDP 是强化学习的标准数学框架，由五元组 $\langle S, A, P, R, \gamma \rangle$ 定义：

- S ：状态空间，表示所有可能的网络状态
- A ：动作空间，表示所有可能的控制动作
- P ：状态转移概率函数， $P(s'|s, a)$ 表示在状态 s 执行动作 a 后转移到状态 s' 的概率
- R ：奖励函数， $R(s, a, s')$ 表示状态转移的即时奖励
- γ ：折扣因子， $0 \leq \gamma \leq 1$ ，表示未来奖励的衰减程度

3.2.1 状态空间设计理论

状态空间的设计是强化学习问题建模的关键环节。一个良好设计的状态空间应当满足以下理论要求：

马尔可夫性 (Markov Property)：当前状态应包含做出最优决策所需的全部信息，即给定当前状态，未来的状态和奖励与过去的状态序列条件独立。数学形式化表示为：

$$P(s_{t+1}, r_{t+1} | s_t, a_t, s_{t-1}, a_{t-1}, \dots, s_0, a_0) = P(s_{t+1}, r_{t+1} | s_t, a_t) \quad (3.1)$$

在网络拥塞控制场景中，完全满足马尔可夫性是困难的，因为网络状态可能依赖于较长的历史。Swift 通过在状态中包含 ECN 状态、CA 状态等协议栈内部状态，在实际应用中近似满足马尔可夫性。

信息充分性 (Information Sufficiency)：状态应包含做出最优决策所需的充分信息。信息不足会导致智能体无法区分需要不同动作的情况，从而影响学习效果。Swift 的 11 维状态空间涵盖了 TCP 协议栈提供的所有关键信息，保证了信息充分性。

维度可扩展性 (Dimensional Scalability)：状态空间的维度不应过高，否则会导致维度灾难 (Curse of Dimensionality) 问题。Swift 的 11 维状态空间在现有方案中属于较高维度，但仍在可处理范围内。基于规则的决策机制避免了维度灾难问题。

可观测性 (Observability)：状态应能够从环境中直接或间接观测获得。Swift 的 15 个状态参数均可从 ns-3 TCP 模块的标准回调接口直接获取，满足可观测性要求。

3.2.2 动作空间设计理论

动作空间的设计决定了智能体能够执行的控制操作范围。Swift 采用连续动作空间，直接输出新的 ssthresh 和 cwnd 值。这种设计的理论考量如下：

控制粒度 (Control Granularity)：连续动作空间提供了最细的控制粒度，允许智能体精确调整控制参数。相比之下，离散动作空间（如“增加 10%”、“减少 10%”）限制了控制的精确性。

探索效率 (Exploration Efficiency)：连续动作空间的探索效率取决于动作的参数化形式。Swift 通过直接输出绝对值而非增量，避免了累积误差问题。

约束满足 (Constraint Satisfaction)：动作应满足 TCP 协议的约束条件。Swift 通过在动作输出后应用边界约束，保证输出的控制参数始终有效。

3.2.3 奖励函数设计理论

奖励函数是强化学习的核心组件，决定了智能体的优化目标。奖励函数设计的理论要求包括：

稀疏性问题 (Sparsity Problem)：如果奖励只在特定事件（如完成文件传输）时给出，智能体可能难以学习中间步骤的价值。Swift 采用即时奖励（每次决策都给出奖励），避免了稀疏性问题。

多目标权衡 (Multi-objective Trade-off)：网络拥塞控制需要平衡多个目标（吞吐量、延迟、公平性、丢包率）。Swift 的奖励函数将吞吐量和延迟结合，通过权重参数控制优化偏好。

奖励塑形 (Reward Shaping)：适当的奖励塑形可以使奖励信号既反映吞吐收益，也反映排队和拥塞代价。当前实现中，正常确认事件的奖励由确认段数带来的吞吐收益减去 RTT 膨胀惩罚组成；ECN、普通丢包和超时丢包分别施加不同强度的负奖励。Python 侧再通过奖励 EMA 参与 α 微调。

3.2.4 策略表示理论

策略 (Policy) 定义了状态到动作的映射。Swift 采用基于规则的确定性策略，而非随机策略或神经网络策略。这种选择的理论考量如下：

确定性 vs. 随机性：确定性策略在给定状态下总是执行相同的动作，具有更好的可预测性和可解释性。随机策略可以提供更好的探索能力，但在部署阶段通常不需要探索。

参数化 vs. 非参数化：神经网络策略是参数化策略，具有强大的表示能力，但训练和推理有计算开销。基于规则的策略是非参数化策略，计算效率高但表示能力有限。Swift 选择基于规则的策略，牺牲部分表示能力换取实时性和可解释性。

在线学习 vs. 离线设计：Swift 的规则是离线设计的，不需要在线学习过程。这避免了在线学习可能带来的不稳定性，但也限制了算法对未知环境的适应能力。

3.3 11 维观测空间详细设计

本节详细描述 Swift 算法的 11 维观测空间设计，这是 Swift 区别于现有方案的核心创新之一。

记法约定：为与 ns-3 侧 OpenGym 实现 (contrib/opengym/examples/swift-tcp/tcp-swift 中 `parameterNum = 15`) 保持一致，每次回调实际通过 Box 跨进程传输的是 15 个元素，其中前 4 个 (socketUuid、envType、simTime、nodeId) 为跨进程路由用的元数据通道，不参与 AIMD/ECN 响应/ α 自适应等窗口决策；后 11 个 (ssthresh、cwnd、segSize、segmentsAcked、

bytesInFlight、lastRtt、minRtt、callbackType、caState、caEvent、ecnState) 构成参与决策的有效 RL 观测子空间。故下文所称“11 维观测空间”特指该有效 RL 子空间；表 3.1 同时列出元数据通道以便审阅者核对完整布局。

表 3.1 Swift 11 维观测空间参数定义

索引	参数名称	数据类型	说明
0	socketUuid	uint64	套接字唯一标识符
1	envType	uint64	环境类型 (0= 事件驱动, 1= 时间驱动)
2	simTime	uint64	仿真时间 (微秒)
3	nodeId	uint64	节点标识符
4	ssthresh	uint64	慢启动阈值 (字节)
5	cwnd	uint64	拥塞窗口大小 (字节)
6	segSize	uint64	TCP 报文段大小 (字节)
7	segmentsAcked	uint64	本次确认的报文段数量
8	bytesInFlight	uint64	在途字节数
9	lastRtt	uint64	最近往返时延 (微秒)
10	minRtt	uint64	最小往返时延 (微秒)
11	callbackType	uint64	回调函数类型
12	caState	uint64	拥塞算法状态
13	caEvent	uint64	拥塞算法事件
14	ecnState	uint64	ECN 状态

3.3.1 连接标识参数 (索引 0、3)

socketUuid (索引 0) 是套接字的唯一标识符, 用于区分不同的 TCP 连接。在多流并发场景下, 每个流需要维护独立的状态信息, socketUuid 是实现每流状态隔离的关键。socketUuid 在连接建立时由系统分配, 在连接生命周期内保持不变。

nodeId (索引 3) 是网络节点的标识符, 用于区分不同的发送端节点。在多节点仿真场景下, 不同节点可能运行不同的拥塞控制算法或具有不同的网络条件, nodeId 用于识别当前决策所属的节点。

3.3.2 时间信息参数 (索引 1、2)

envType (索引 1) 是环境类型标识符。当前 OpenGym 事件驱动实现中该字段固定为 0, 用于标识观测来自 TCP 回调事件路径。

simTime (索引 2) 是当前仿真时间, 单位为微秒。仿真时间用于标识观测发生的事件顺序, 并为跨进程观测和动作交互提供时间上下文。

3.3.3 窗口状态参数（索引 4、5）

ssthresh（索引 4）是慢启动阈值（Slow Start Threshold），单位为字节。**ssthresh** 是 TCP 拥塞控制的核心状态变量之一，用于区分慢启动阶段和拥塞避免阶段：当 $cwnd < ssthresh$ 时处于慢启动阶段，窗口指数增长；当 $cwnd \geq ssthresh$ 时进入拥塞避免阶段，窗口线性增长。

cwnd（索引 5）是拥塞窗口（Congestion Window）大小，单位为字节。**cwnd** 是 TCP 拥塞控制的另一核心状态变量，决定了发送端在未收到确认的情况下最多可以发送的数据量。**cwnd** 的调整是拥塞控制的核心任务。

3.3.4 传输性能参数（索引 6、7、8）

segSize（索引 6）是 TCP 报文段大小（Maximum Segment Size, MSS），单位为字节。典型值为 1460 字节（以太网 MTU 1500 字节减去 IP 报头 20 字节和 TCP 报头 20 字节）。**segSize** 用于将字节为单位的度量转换为报文段为单位的度量。

segmentsAcked（索引 7）是本次确认的报文段数量。在收到累积确认（Cumulative ACK）或选择性确认（Selective ACK, SACK）时，**segmentsAcked** 表示新确认的报文段数量。**segmentsAcked** 用于计算窗口增长量，例如在慢启动阶段，窗口增长量为 $segmentsAcked \times MSS$ 。

bytesInFlight（索引 8）是在途字节数（Bytes in Flight），即已发送但尚未确认的数据量。**bytesInFlight** 用于计算管道利用率（Pipeline Utilization）：

$$utilization = \frac{bytesInFlight}{cwnd} \quad (3.2)$$

当 **utilization** 较低时，表明发送端未能充分利用拥塞窗口，可能是因为应用层数据不足或接收端通告窗口限制。

3.3.5 延迟测量参数（索引 9、10）

lastRtt（索引 9）是最近测量的往返时延（Round-Trip Time），单位为微秒。**RTT** 是端到端延迟的直接度量，包括传播延迟（Propagation Delay）、排队延迟（Queuing Delay）、处理延迟（Processing Delay）和传输延迟（Transmission Delay）四个组成部分。

minRtt（索引 10）是历史最小往返时延，单位为微秒。**minRtt** 作为路径传播时延的保守下界，用于估计带宽延迟积（BDP）和计算 **RTT** 膨胀程度。**RTT** 膨胀比定义为：

$$inflation_ratio = \frac{lastRtt}{minRtt} \quad (3.3)$$

当 `inflation_ratio` 接近 1 时, 表明网络排队延迟较小; 当 `inflation_ratio` 较大时, 表明网络存在显著的排队延迟。

3.3.6 回调类型参数 (索引 11)

callbackType (索引 11) 标识当前回调的类型, 用于区分不同类型的事件。定义如下:

- 值为 0: `GetSsThresh` 回调, 表示检测到丢包事件, 需要更新慢启动阈值
- 值为 1: `IncreaseWindow` 回调, 表示收到正常确认, 可以增加拥塞窗口

`callbackType` 是多信号融合拥塞检测的关键输入之一。当 `callbackType` 为 0 时, 可以明确判断发生了丢包, 这是最高优先级的拥塞信号。

3.3.7 拥塞控制状态参数 (索引 12、13、14)

caState (索引 12) 是拥塞算法状态 (Congestion Algorithm State), 反映 TCP 拥塞控制状态机的当前状态。Linux TCP 实现定义了以下状态:

- `CA_OPEN` (值为 0): 开放状态, 正常传输, 无拥塞迹象
- `CA_DISORDER` (值为 1): 无序状态, 收到重复确认或乱序确认
- `CA_CWR` (值为 2): 拥塞窗口缩减状态, 响应 ECN 标记
- `CA_RECOVERY` (值为 3): 恢复状态, 快速恢复过程中
- `CA_LOSS` (值为 4): 丢失状态, 检测到超时丢包

caEvent (索引 13) 是拥塞算法事件 (Congestion Algorithm Event), 表示最近发生的拥塞相关事件。Linux TCP 实现定义了以下事件:

- `CA_EVENT_TX_START` (值为 0): 传输开始事件
- `CA_EVENT_CWND_RESTART` (值为 1): 拥塞窗口重启事件
- `CA_EVENT_COMPLETE_CWR` (值为 2): CWR 完成事件
- `CA_EVENT_LOSS` (值为 3): 丢失事件
- `CA_EVENT_ECN_NO_CE` (值为 4): 无 CE 标记的 ECN 事件

- `CA_EVENT_ECN_IS_CE` (值为 5): 有 CE 标记的 ECN 事件

ecnState (索引 14) 是显式拥塞通知状态 (ECN State), 反映 ECN 协商和标记的状态。定义如下:

- `ECN_DISABLED` (值为 0): ECN 禁用
- `ECN_IDLE` (值为 1): ECN 空闲
- `ECN_CE_RCVD` (值为 2): 收到 CE 标记
- `ECN_SENDING_ECE` (值为 3): 正在发送 ECE 标志
- `ECN_ECE_RCVD` (值为 4): 收到 ECE 标志
- `ECN_CWR_SENT` (值为 5): 已发送 CWR 标志

这三个拥塞控制状态参数是 Swift 观测空间的核心创新。现有强化学习拥塞控制方案通常不包含这些协议栈内部状态, 导致智能体对网络拥塞状况的感知不完整。Swift 通过将 `caState`、`caEvent`、`ecnState` 纳入观测空间, 使智能体能够感知 TCP 协议栈的完整状态信息, 从而做出更精准的控制决策。

3.4 多信号融合拥塞检测算法

Swift 算法的核心创新之一是多信号融合拥塞检测方法。该方法综合利用丢包回调、ECN 状态和 TCP 拥塞状态机信息, 采用显式信号优先级判断拥塞类型, 并将 RTT 膨胀保留给参数自适应调节, 而不把 RTT 膨胀作为独立降窗触发条件。

3.4.1 拥塞信号分类

Swift 将拥塞信号分为四个类别:

(1) **显式丢包信号**: 当 `callbackType` 为 0 (`GetSsThresh` 回调) 时, 表明检测到了丢包事件。丢包是最明确的拥塞信号, 通常意味着网络缓冲区已经溢出。

(2) **ECN 拥塞经历信号**: 当 `ecnState` 为 `ECN_CE_RCVD` (值为 2) 或 `caEvent` 为 `CA_EVENT_ECN_IS_CE` (值为 5) 时, 表明数据包被网络设备标记为拥塞经历。ECN 信号是早期拥塞信号, 表明网络设备的队列已经超过阈值, 但尚未溢出。

(3) **拥塞控制状态机信号**: 当 `caState` 为 `CA_CWR`、`CA_RECOVERY` 或 `CA_LOSS` 时, 表明 TCP 协议栈的拥塞控制状态机已经检测到拥塞迹象并进入相应状态。

(4) **延迟膨胀信号**: 当 RTT 膨胀比 ($\text{lastRtt}/\text{minRtt}$) 超过阈值时, 表明网络存在显著的排队延迟, 可能处于拥塞状态。

3.4.2 分层优先级机制

不同类型的拥塞信号具有不同的可靠性和紧迫程度，Swift 采用分层优先级机制进行处理：

第一优先级：显式丢包检测

Algorithm 1 显式丢包检测

Require: 回调类型 *callbackType*, 拥塞算法状态 *caState*

```

1: if callbackType = 0 then                                ▷ GetSsThresh 回调
2:   is_congested  $\leftarrow$  True
3:   if caState = CA_LOSS then
4:     congestion_type  $\leftarrow$  TIMEOUT
5:   else
6:     congestion_type  $\leftarrow$  LOSS
7:   end if
8:   return (is_congested, congestion_type)
9: end if

```

当 *callbackType* 为 0 时，明确判定发生丢包。Swift 进一步区分快速重传检测到的丢包 (*caState* $\neq$ *CA_LOSS*) 和超时导致的丢包 (*caState* = *CA_LOSS*)，分别采用不同的响应策略。这是最高优先级的拥塞信号，一旦检测到丢包，无需进一步检查其他信号。

第二优先级：ECN 拥塞经历信号检测

Algorithm 2 ECN 拥塞经历信号检测

Require: ECN 状态 *ecnState*

```

1: if ecnState  $\in$  {ECN_CE_RCVD, ECN_ECE_RCVD} then
2:   is_congested  $\leftarrow$  True
3:   congestion_type  $\leftarrow$  ECN
4:   return (is_congested, congestion_type)
5: end if

```

当 *ecnState* 为 *ECN_CE_RCVD* (收到 CE 标记) 或 *ECN_ECE_RCVD* (收到 ECE 回显) 时，判定发生 ECN 拥塞。ECN 拥塞采用早期预警响应策略，当前实现的保留因子为 0.75；该值比普通丢包响应更温和，同时能够更及时释放队列压力。

3.4.3 信号优先级与误判抑制策略

Swift 的拥塞检测采用显式信号优先级，而不是将所有协议栈状态都作为独立拥塞触发条件。当前实现中，显式丢包回调具有最高优先级；ECN CE/ECE 状态是第二优先级的早期拥塞信号；RTT 膨胀、*CA_CWR* 和 *CA_RECOVERY* 等状态不直接触发降窗，而用于解释网络状态或影响后续参数调节。

(1) **显式丢包优先**: 当回调类型为 `GetSsThresh` 时, 算法认为 TCP 协议栈已经进入丢包相关处理路径, 并根据 CA 状态区分普通丢包和超时丢包。

(2) **ECN 直接触发早期响应**: 当 ECN 状态为 `ECN_CE_RCVD` 或 `ECN_ECE_RCVD` 时, 算法将其视为有效早期拥塞信号并触发 ECN 响应, 同时累加 ECN 计数器用于后续状态分析。

(3) **RTT 膨胀不单独触发降窗**: RTT 膨胀容易受到无线调度、ACK 压缩和路径抖动影响。当前实现将其用于 α 自适应调节和慢启动退出判断, 而不是作为独立拥塞事件直接执行乘性递减。

(4) **降窗后冻结抑制反弹**: 每次显式拥塞响应后, 算法冻结后续若干 ACK 事件中的窗口增长, 避免刚降窗后立即增窗造成二次排队。

3.4.4 完整拥塞检测算法

综合上述机制, Swift 的完整多信号融合拥塞检测算法如下:

Algorithm 3 多信号融合拥塞检测算法

Require: 观测状态 *obs*, 流状态 *state*

Ensure: 是否拥塞 *is_congested*, 拥塞类型 *type*

```

1:
2: if obs.callbackType = 0 then
3:   state.loss_count  $\leftarrow$  state.loss_count + 1
4:   if obs.caState = CA_LOSS then
5:     return (True, TIMEOUT)
6:   else
7:     return (True, LOSS)
8:   end if
9: end if
10:
11: if obs.ecnState  $\in$  {ECN_CE_RCVD, ECN_ECE_RCVD} then
12:   state.ecn_count  $\leftarrow$  state.ecn_count + 1
13:   return (True, ECN)
14: end if
15:
16: return (False, NONE)
```

▷ 第一优先级: 显式丢包检测
▷ GetSsThresh 回调
▷ 超时丢包
▷ 快速重传丢包
▷ 第二优先级: ECN 拥塞经历检测
▷ 未检测到拥塞

3.5 差异化拥塞响应机制

Swift 算法针对不同类型的拥塞信号采用差异化的响应策略, 核心是差异化窗口保留因子 (Window Retention Factor) 机制。

3.5.1 窗口保留因子设计

窗口保留因子 ρ 定义为拥塞响应后新窗口与原窗口的比值：

$$new_cwnd = \rho \cdot cwnd$$

(3.4)

传统算法（如 TCP Reno、CUBIC）对所有拥塞信号采用统一的窗口保留因子（通常为 0.5，即窗口减半）。这种”一刀切”的策略无法区分不同类型和程度的拥塞，在轻度拥塞时响应过度，在严重拥塞时响应不足。

Swift 根据拥塞类型设计了差异化的窗口保留因子：

表 3.2 差异化窗口保留因子

拥塞类型	保留因子	设计理由
丢包（Loss）	0.70	丢包是严重拥塞信号，需要较大幅度降低窗口
ECN 标记（ECN）	0.75	ECN 是早期信号，提前释放队列压力
超时丢失（Timeout）	0.50	超时表示最严重拥塞，需最大幅度窗口缩减

3.5.2 差异化响应的理论依据

差异化窗口保留因子的设计基于以下理论分析：

(1) 拥塞信号的时序特性

ECN 标记是最早的拥塞信号，通常在队列长度超过低阈值时触发，此时网络可能只是轻度拥塞。丢包是较晚的拥塞信号，通常在队列溢出时发生，表明网络已经严重拥塞。超时丢包是最晚也最严重的拥塞信号，通常意味着多个数据包连续丢失或网络路径严重中断，需要最大力度的窗口缩减。

根据时序关系和严重程度，Swift 设计了三级差异化响应：ECN 标记采用早期预警响应（保留因子 0.75），丢包采用较激进的响应（保留因子 0.70），超时采用最激进的响应（保留因子 0.50）。这种阶梯式的响应力度设计用于匹配不同严重程度的拥塞信号，同时通过降窗后冻结机制抑制窗口立即反弹。

(2) 信号可靠性分析

丢包是最可靠的拥塞信号，几乎不会出现误报（False Positive）。ECN 标记的可靠性取决于网络设备的配置和标记阈值。如果阈值设置不当，可能出现过早或过晚标记。基于可靠性考虑，对丢包信号的响应应更加坚决；对 ECN 信号的响应既不能等同于丢包，也不能过于宽松。当前实现采用 0.75 的 ECN 保留因子，在早期释放队列压力和维持吞吐量之间取得折中。

(3) 吞吐量-延迟权衡

较低的窗口保留因子能够更快地缓解拥塞、降低延迟，但会导致吞吐量下降。较高的窗口保留因子能够维持较高的吞吐量，但可能延长拥塞持续时间。Swift 的差异化设计试图在吞吐量和延迟之间取得最优平衡。

3.5.3 慢启动阈值更新

除了拥塞窗口，拥塞响应还需要更新慢启动阈值。当前实现首先以 $\rho \times cwnd$ 计算新的拥塞窗口，并设置最小窗口边界 $W_{min} = \max(4 \times MSS, 1)$ 。慢启动阈值采用 BDP 锚定但不奖励队列膨胀的策略：

$$ref = \min(cwnd, \max(BDP, W_{min})) \quad (3.5)$$

$$new_ssthresh = \max(\rho \times ref, W_{min}, new_cwnd) \quad (3.6)$$

该设计避免在队列已膨胀时把慢启动阈值设置到过高水平，同时保证阈值不低于新的拥塞窗口，防止拥塞恢复阶段出现窗口停滞。

此外，Swift 引入连续缩减保护和降窗后冻结机制。每次拥塞响应后连续缩减计数器递增；当连续缩减次数超过 3 次时，算法保持当前窗口不再继续缩减，避免窗口死亡螺旋。任一显式降窗后，算法冻结后续 4 个 ACK 事件中的窗口增长，使队列有时间排空。

3.5.4 差异化拥塞响应算法

3.6 强化学习驱动的自适应参数调优

Swift 算法实现了强化学习驱动的自适应参数调优机制，根据多种因素动态调整乘性增加因子（Multiplicative Increase Factor, α ）。

3.6.1 乘性增加因子的作用

在 Swift 的融合控制策略中，乘性增加因子 α 控制拥塞避免阶段的窗口增长速度：

$$target_ref = \alpha \times BDP + \gamma \times MSS \quad (3.7)$$

α 的取值决定了窗口围绕 BDP 目标的激进程度：较大的 α 使目标窗口更接近或略高于估计 BDP，能够更快地利用可用带宽，但也更容易形成排队；较小的 α 使目标窗口低于或接近 BDP，有利于排队释放和延迟恢复。

Algorithm 4 差异化拥塞响应算法**Require:** 当前窗口 $cwnd$, 当前阈值 $ssthresh$, 拥塞类型 $type$, 连续缩减计数 $consec_dec$ **Ensure:** 新窗口 new_cwnd , 新阈值 $new_ssthresh$

```

1:  $consec\_dec \leftarrow consec\_dec + 1$ 
2:                                     ▷ 连续缩减保护：超过 3 次后保持窗口
3: if  $consec\_dec > 3$  then
4:     return ( $cwnd, cwnd$ )                                     ▷ 保持当前窗口不再缩减
5: end if
6:                                     ▷ 根据拥塞类型选择窗口保留因子
7: if  $type = TIMEOUT$  then
8:      $\rho \leftarrow 0.50$                                          ▷ 超时：最严厉响应
9: else if  $type = LOSS$  then
10:     $\rho \leftarrow 0.70$                                          ▷ 丢包：较激进响应
11: else if  $type = ECN$  then
12:     $\rho \leftarrow 0.75$                                          ▷ ECN：早期预警响应
13: else
14:     $\rho \leftarrow 1.0$                                          ▷ 无拥塞，不调整
15: end if
16:                                     ▷ 计算新窗口
17:  $new\_cwnd \leftarrow \lfloor \rho \times cwnd \rfloor$ 
18:                                     ▷ 应用窗口下限约束
19:  $new\_cwnd \leftarrow \max(new\_cwnd, 4 \times MSS)$ 
20:                                     ▷ 计算新慢启动阈值：以 BDP 和当前窗口的较小有效参考为锚定
21:  $ref \leftarrow \min(cwnd, \max(bdp, 4 \times MSS))$ 
22:  $new\_ssthresh \leftarrow \max(\lfloor \rho \times ref \rfloor, new\_cwnd)$ 
23: return ( $new\_cwnd, new\_ssthresh$ )

```

3.6.2 多因素调整机制

Swift 根据以下因素动态调整 α :

(1) 基于 RTT 膨胀比的路径感知调整

RTT 膨胀比反映网络排队延迟程度:

$$inflation_ratio = \frac{lastRtt}{minRtt} \quad (3.8)$$

调整规则采用与最小 RTT 相关的动态阈值。首先根据最小 RTT 计算松弛量:

$$s = 0.3 + 0.15\sqrt{\max(0, RTT_{min,ms} - 1)} \quad (3.9)$$

并构造 $1 + s$ 、 $1 + 2s$ 和 $1 + 4s$ 三个路径感知阈值。短 RTT 路径对排队更敏感，阈值更紧；长 RTT 路径允许更大的正常抖动，阈值更宽。具体调整规则为:

- 当 $inflation_ratio < 1 + s$ 时，认为排队较小，增大 α ；若最小 RTT 超过 5 ms，则增长步长为 0.02，否则为 0.03。

- 当 $1 + s \leq inflation_ratio < 1 + 2s$ 时, 认为排队适中, 温和增大 α : $\alpha \leftarrow \alpha + 0.01$ 。
- 当 $inflation_ratio > 1 + 4s$ 时, 认为排队较重, 减小 α ; 若最小 RTT 超过 5 ms, 则减小步长为 0.03, 否则为 0.02。

这种路径感知响应机制能够避免固定阈值在远距离路径上过早收紧窗口, 同时保留低 RTT 路径中对排队增长的敏感性。

(2) 基于奖励信号的趋势调整

Swift 利用环境反馈的奖励信号 (Reward) 进行策略效果评估。采用指数移动平均 (Exponential Moving Average, EMA) 对奖励信号进行平滑处理:

$$R_{ema} \leftarrow (1 - \eta) \times R_{ema} + \eta \times reward \quad (3.10)$$

其中 EMA 平滑系数为 0.15。根据奖励趋势调整 α :

- 当 $R_{ema} > 0.5$ 时, 表明当前策略效果较好, 增大 α : $\alpha \leftarrow \alpha + 0.01$
- 当 $R_{ema} < -2.0$ 时, 表明当前策略效果不佳, 减小 α : $\alpha \leftarrow \alpha - 0.01$

奖励信号的引入使得 Swift 能够从全局控制效果的角度评估参数调整的方向, 弥补了仅依赖 RTT 膨胀比等局部指标的不足。

(3) 基于连续增长趋势的调整

当窗口连续增长超过 8 次时, 表明网络状况持续良好, 给予稳定增长奖励:

$$\alpha \leftarrow \alpha + 0.01 \quad \text{如果连续增长次数} > 8 \quad (3.11)$$

3.6.3 参数边界约束

为保证控制策略的稳定性, α 的取值被限制在预设范围内:

$$\alpha \in [\alpha_{min}, \alpha_{max}] = [0.85, 1.30] \quad (3.12)$$

α 的默认初始值为 1.10。下界 0.85 为无线、蜂窝或浅缓冲路径提供 stronger 的排队释放空间; 上界 1.30 限制窗口相对于 BDP 的长期扩张幅度, 避免慢性队列积累。

Algorithm 5 自适应参数调优算法**Require:** 当前 α , 观测状态 obs , 奖励信号 $reward$, 历史状态 $history$ **Ensure:** 更新后的 α

```

1:                                     ▷ 因素 1: 基于 RTT 膨胀比的路径感知调整
2:  $inflation\_ratio \leftarrow obs.lastRtt / obs.minRtt$ 
3:  $min\_rtt\_ms \leftarrow obs.minRtt / 1000$ 
4:  $slack \leftarrow 0.3 + 0.15\sqrt{\max(0, min\_rtt\_ms - 1)}$ 
5:  $low \leftarrow 1 + slack, mid \leftarrow 1 + 2 \times slack, high \leftarrow 1 + 4 \times slack$ 
6: if  $inflation\_ratio < low$  then
7:    $ramp \leftarrow 0.02$  if  $min\_rtt\_ms > 5$  else  $0.03$ 
8:    $\alpha \leftarrow \alpha + ramp$ 
9:    $history.consecutive\_growth \leftarrow history.consecutive\_growth + 1$ 
10: else if  $inflation\_ratio < mid$  then
11:    $\alpha \leftarrow \alpha + 0.01$ 
12: else if  $inflation\_ratio > high$  then
13:    $step \leftarrow 0.03$  if  $min\_rtt\_ms > 5$  else  $0.02$ 
14:    $\alpha \leftarrow \alpha - step$ 
15:    $history.consecutive\_growth \leftarrow 0$ 
16: end if
17:                                     ▷ 因素 2: 基于奖励信号 EMA 的趋势调整
18:  $R_{ema} \leftarrow 0.85 \times R_{ema} + 0.15 \times reward$ 
19: if  $R_{ema} > 0.5$  then
20:    $\alpha \leftarrow \alpha + 0.01$ 
21: else if  $R_{ema} < -2.0$  then
22:    $\alpha \leftarrow \alpha - 0.01$ 
23: end if
24:                                     ▷ 因素 3: 基于连续稳定增长的奖励
25: if  $history.consecutive\_growth > 8$  then
26:    $\alpha \leftarrow \alpha + 0.01$ 
27: end if
28:                                     ▷ 应用边界约束
29:  $\alpha \leftarrow \max(\min(\alpha, 1.30), 0.85)$ 
30: return  $\alpha$ 

```

3.6.4 自适应参数调优算法

3.7 融合控制策略

Swift 采用融合控制策略计算目标拥塞窗口，将基于带宽延迟积（BDP）估计的速率控制与基于当前窗口的保守控制相结合。

3.7.1 带宽延迟积估计

带宽延迟积（Bandwidth-Delay Product, BDP）是网络链路能够容纳的最大在途数据量，是拥塞控制的重要参考指标：

$$BDP = Bandwidth \times RTT \quad (3.13)$$

Swift 通过以下方式估计 BDP：

$$BDP = \max_{i \in W} (delivery_rate_i) \times minRtt \quad (3.14)$$

其中， $\max_{i \in W} (delivery_rate_i)$ 是窗口化最大值过滤（Windowed Max-filter）方法估计的瓶颈带宽，窗口大小 $W = 20$ 个样本点。单个样本的投递率（Delivery Rate）计算为：

$$delivery_rate = \frac{segmentsAked \times segSize}{lastRtt} \quad (3.15)$$

相比简单地使用历史最大吞吐量，窗口化最大值过滤方法的优势在于：（1）通过限制采样窗口大小，能够适应网络带宽的动态变化；（2）取窗口内最大值能够过滤 RTT 波动导致的低估；（3）计算复杂度为 $O(1)$ （每次插入新样本时更新最大值）。

吞吐量的跟踪采用指数移动平均方式：

$$throughput_ema = 0.9 \times throughput_ema + 0.1 \times delivery_rate \quad (3.16)$$

3.7.2 慢启动阶段策略

在慢启动阶段（ $cwnd < ssthresh$ ），Swift 采用指数增长策略以快速探测可用带宽：

$$target_cwnd = 2 \times BDP \quad (3.17)$$

$$increment = segmentsAcked \times MSS \quad (3.18)$$

当窗口远低于 BDP 时，采用加速增长：

$$\text{如果 } cwnd < 0.3 \times BDP, \quad increment = 2 \times segmentsAcked \times MSS \quad (3.19)$$

慢启动目标设为 $2 \times BDP$ 而非更大值（如 $3 \times BDP$ ），是为了避免慢启动阶段过度膨胀导致突发丢包。当窗口达到目标值后，自动退出慢启动进入拥塞避免阶段。

3.7.3 拥塞避免阶段策略

在拥塞避免阶段（ $cwnd \geq ssthresh$ ），Swift 采用稳健的窗口增长策略：

$$target_ref = \alpha \times BDP + \gamma \times MSS \quad (3.20)$$

其中， α 是乘性增加因子（通过自适应调优动态调整，基准值 1.10）， $\gamma = 1.0$ 是加性增加因子，表示每个 ACK 事件引入 1 个 MSS 的基础增量。拥塞避免阶段不再简单地以当前窗口作为下界持续取最大值，而是围绕 $\alpha \times BDP$ 目标进行拉升或回落：当目标高于当前窗口时，以 $\max(\gamma \times MSS, MSS, gap/16)$ 的步长向目标靠近；当目标低于当前窗口时，每个 ACK 事件按超出量的一半进行回落，以更快释放浅缓冲路径中的排队。

融合控制策略还考虑管道利用率因素。管道利用率定义为：

$$utilization = \frac{bytesInFlight}{cwnd} \quad (3.21)$$

调整规则：

- 仅当 RTT 未明显膨胀（ $lastRtt < 1.3 \times minRtt$ ）时启用利用率补偿。
- 当 $utilization < 0.4$ 时，短 RTT 路径额外增加 $2 \times MSS$ ，长 RTT 路径额外增加 $1 \times MSS$ 。
- 当 $0.4 \leq utilization < 0.5$ 且路径不是长 RTT 路径时，额外增加 $1 \times MSS$ 。

该设计只在管道确实未充分利用且队列未增长时加速，避免旧式利用率补偿在无线、蜂窝或浅缓冲路径中造成持续排队。

3.7.4 窗口边界约束

为保证窗口大小的合理性，Swift 对计算结果应用边界约束：

$$cwnd \in [4 \times MSS, \max(4 \times BDP, 200 \times MSS)] \quad (3.22)$$

下界 $4 \times MSS$ 保证基本传输能力。上界设为 BDP 的 4 倍或 200 个 MSS 的较大值，用于在 BDP 估计暂时偏大或路径缓冲较浅时限制窗口长期膨胀。

3.7.5 融合控制策略算法

3.8 每流独立状态管理

Swift 设计了每流独立的状态管理机制，为每个 TCP 连接维护独立的状态数据结构，支持多流并发场景下的差异化控制。

3.8.1 状态数据结构

每个 TCP 连接维护以下状态数据，采用层次化组织方式：

(1) 历史度量缓冲区：带宽估计采用窗口化最大值过滤，维护一个容量为 40 个样本点的投递率 (Delivery Rate) 滑动窗口队列，取窗口内最大值作为瓶颈带宽估计。RTT 跟踪维护一个容量为 40 个样本点的最近 RTT 队列，同时维护全局最小 RTT 作为传播延迟估计。BDP 由最大带宽与最小 RTT 的乘积得到。

(2) 拥塞事件计数数据：状态结构维护丢包事件计数和 ECN 标记事件计数，用于记录每条流在仿真过程中的拥塞事件分布。

(3) 性能指标数据：性能指标模块维护吞吐量的指数移动平均值 (EMA，平滑因子 0.1)，用于跟踪流级别的性能变化趋势。

(4) 自适应参数数据：自适应参数模块维护乘性增加因子 α (取值范围 $[0.85, 1.30]$ ，默认值 1.10)、加性增加因子 γ (默认值 1.0)、连续缩减计数器 (用于连续缩减保护)、连续增长计数器 (用于稳定增长奖励)、降窗后冻结 ACK 计数器 (默认 4 个 ACK 事件) 和慢启动阶段标记。

3.8.2 状态查找与更新

Swift 使用 socketUuid 作为键，通过哈希表实现 $O(1)$ 时间复杂度的状态查找和更新。哈希表的选择基于查找效率、动态扩展和内存效率三方面考量。

Algorithm 6 融合控制策略算法**Require:** 观测状态 *obs*, 自适应参数 α , $\gamma = 1.0$, BDP 估计 *bdp***Ensure:** 新窗口 *new_cwnd*

```

1:  $\triangleright$  降窗后冻结: 显式降窗后的若干 ACK 事件内保持窗口不增长
2: if state.freeze_acks_remaining > 0 then
3:   state.freeze_acks_remaining  $\leftarrow$  state.freeze_acks_remaining - 1
4:   return obs.ssthresh, obs.cwnd
5: end if
6:  $\triangleright$  计算管道利用率
7: utilization  $\leftarrow$  obs.bytesInFlight / obs.cwnd
8:  $\triangleright$  判断当前阶段
9: if obs.cwnd < obs.ssthresh and in_slow_start then
10:  $\triangleright$  HyStart 风格慢启动退出
11:   if obs.lastRtt > HystartThreshold(state)  $\times$  state.ss_min_rtt_sample then
12:     state.in_slow_start  $\leftarrow$  False
13:     return  $\max(\text{obs.ssthresh}, \text{obs.cwnd})$ , obs.cwnd
14:   end if
15:   target  $\leftarrow$   $\max(2 \times \text{bdp}, 10 \times \text{obs.segSize})$ 
16:   increment  $\leftarrow$  obs.segmentsAked  $\times$  obs.segSize
17:   if obs.cwnd <  $0.3 \times \text{bdp}$  and obs.lastRtt <  $1.2 \times \text{state.min\_rtt}$  then
18:     increment  $\leftarrow$   $2 \times \text{obs.segmentsAked} \times \text{obs.segSize}$ 
19:   end if
20:   new_cwnd  $\leftarrow$   $\min(\text{obs.cwnd} + \text{increment}, \text{target})$ 
21: else
22:  $\triangleright$  拥塞避免阶段: 围绕  $\alpha \times \text{BDP}$  目标进行有界步长拉升或回落
23:   state.in_slow_start  $\leftarrow$  False
24:   target_rate  $\leftarrow$   $\lfloor \alpha \times \text{bdp} \rfloor$ 
25:   gamma_bytes  $\leftarrow$   $\gamma \times \text{obs.segSize}$ 
26:   if target_rate > obs.cwnd then
27:     gap  $\leftarrow$  target_rate - obs.cwnd
28:     step  $\leftarrow$   $\max(\text{gamma\_bytes}, \text{obs.segSize}, \text{gap}/16)$ 
29:     new_cwnd  $\leftarrow$   $\min(\text{obs.cwnd} + \text{step}, \text{target\_rate} + \text{gamma\_bytes})$ 
30:   else if target_rate < obs.cwnd then
31:     excess  $\leftarrow$  obs.cwnd - target_rate
32:     new_cwnd  $\leftarrow$   $\max(\text{obs.cwnd} - \max(\text{excess}/2, \text{obs.segSize}), \text{target\_rate})$ 
33:   else
34:     new_cwnd  $\leftarrow$  obs.cwnd + gamma_bytes
35:   end if
36:  $\triangleright$  仅在 RTT 未膨胀时启用管道利用率补偿; 长 RTT 路径降低补偿强度
37:   if obs.lastRtt <  $1.3 \times \text{state.min\_rtt}$  and utilization < 0.4 then
38:     new_cwnd  $\leftarrow$  new_cwnd + (state.min_rtt > 5ms ? obs.segSize :  $2 \times \text{obs.segSize}$ )
39:   else if obs.lastRtt <  $1.3 \times \text{state.min\_rtt}$  and utilization < 0.5 and state.min_rtt  $\leq$  5ms
40:   then
41:     new_cwnd  $\leftarrow$  new_cwnd + obs.segSize
42:   end if
43: end if
44: return obs.ssthresh, new_cwnd

```

Algorithm 7 流状态查找与更新

```

1: function LookupFlowState(manager, socket_uuid)
2:   hash  $\leftarrow$  ComputeHash(socket_uuid)
3:   bucket  $\leftarrow$  manager.buckets[hash mod manager.num_buckets]
4:   for each entry in bucket do
5:     if entry.key = socket_uuid then
6:       return entry.value
7:     end if
8:   end for
9:   return null
10: end function
11: function UpdateFlowState(manager, socket_uuid, obs)
12:   state  $\leftarrow$  GetOrCreateFlowState(manager, socket_uuid)
13:   state.last_update_time  $\leftarrow$  obs[2]
14:   UpdateMetrics(state.metrics_buffer, obs)
15:   if IsECNEvent(obs) then
16:     state.ecn_tracker.RecordEvent(obs[2])
17:   end if
18:   UpdatePerformanceMetrics(state.performance, obs)
19: end function

```

▷ 未找到

3.8.3 状态生命周期管理

TCP 连接的生命周期包括建立、传输和关闭三个阶段。Swift 的状态管理遵循延迟初始化、增量更新、垃圾回收和状态持久化四个原则。

Algorithm 8 流状态垃圾回收

```

1: function GarbageCollect(manager, current_time, inactive_threshold)
2:   to_remove  $\leftarrow$  EmptyList()
3:   for each (uuid, state) in manager.flow_states do
4:     idle_time  $\leftarrow$  current_time - state.last_update_time
5:     if idle_time > inactive_threshold then
6:       to_remove.Append(uuid)
7:     end if
8:   end for
9:   for each uuid in to_remove do
10:    manager.flow_states.Remove(uuid)
11:    FreeResources(uuid)
12:   end for
13:   return |to_remove|
14: end function

```

3.9 实现细节与优化技术

本节详细介绍 Swift 算法的具体实现细节和优化技术。

3.9.1 系统架构设计

Swift 算法的系统架构遵循模块化设计原则，采用分层结构实现各功能模块的解耦。整体架构分为仿真层、环境层、决策层和通信层四个层次。仿真层负责网络拓扑构建和数据包传输仿真；环境层作为强化学习与网络仿真的桥梁；决策层实现 Swift 算法的核心逻辑，遵循状态隔离、分层决策和实时性原则；通信层负责仿真进程和决策进程之间的数据交换。

Algorithm 9 Swift 系统初始化流程

Require: 网络拓扑配置 *topology*, 仿真参数 *params*

Ensure: 初始化完成的 Swift 系统

```

1: simulator  $\leftarrow$  CreateSimulator(params.duration)
2: network  $\leftarrow$  BuildTopology(topology)
3: ConfigureQueues(network, params.queue_size)
4: env  $\leftarrow$  CreateOpenGymEnvironment()
5: RegisterCallbacks(env, network.tcp_sockets)
6: ConfigureObservationSpace(env, 15)
7: ConfigureActionSpace(env, 2) ▷ ssthresh 和 cwnd
8: agent  $\leftarrow$  CreateSwiftAgent()
9: agent.flow_states  $\leftarrow$  EmptyHashMap()
10: agent.alpha_default  $\leftarrow$  1.10
11: channel  $\leftarrow$  CreateMessageChannel()
12: ConnectEnvAndAgent(env, agent, channel)
13: simulator.Run()

```

3.9.2 核心算法伪代码

Swift 智能体的主控制循环负责协调各功能模块，处理来自环境的观测并输出控制动作。

3.9.3 数据结构设计

Swift 算法采用高效的数据结构支持其核心功能。**循环队列 (Circular Buffer)** 用于存储带宽与 RTT 历史样本，具有固定内存占用、 $O(1)$ 时间复杂度的插入和删除操作、自动保持时间局部性等优势。**每流状态字典**以 socketUuid 为键隔离不同 TCP 连接的 α 、BDP、拥塞事件计数、连续缩减计数和降窗后冻结计数，避免不同流之间的控制状态相互污染。

3.9.4 性能优化技术

为了提高 Swift 算法的执行效率，满足实时决策的要求，本文采用了多种性能优化技术：

- (1) **延迟初始化策略**：流状态采用延迟初始化策略，只在首次访问时创建状态数据结构。
- (2) **增量计算优化**：吞吐量、BDP 等指标的计算采用增量方式，时间复杂度为 $O(1)$ 。

Algorithm 10 Swift 主控制循环**Require:** 环境接口 env , 最大迭代次数 $max_iterations$ **Ensure:** 控制结果序列

```

1:  $iteration \leftarrow 0$ 
2:  $flow\_manager \leftarrow \text{InitFlowStateManager}()$ 
3: while  $iteration < max\_iterations$  and not  $env.IsDone()$  do
4:    $obs \leftarrow env.GetObservation()$ 
5:    $socket\_uuid \leftarrow obs[0]$ 
6:    $flow\_state \leftarrow flow\_manager.GetOrCreate(socket\_uuid)$ 
7:    $\text{UpdateFlowState}(flow\_state, obs)$ 
8:    $(is\_congested, cong\_type, severity) \leftarrow \text{DetectCongestion}(obs, flow\_state)$ 
9:   if  $is\_congested$  then
10:     $action \leftarrow \text{CongestionResponse}(obs, cong\_type, flow\_state)$ 
11:   else
12:     $\text{UpdateAdaptiveParams}(flow\_state, obs)$ 
13:     $action \leftarrow \text{FusionControl}(obs, flow\_state)$ 
14:   end if
15:    $env.ExecuteAction(action)$ 
16:    $iteration \leftarrow iteration + 1$ 
17: end while

```

Algorithm 11 延迟初始化状态获取

```

1: function  $\text{GetOrCreateFlowState}(manager, socket\_uuid)$ 
2:   if  $socket\_uuid \notin manager.flow\_states$  then
3:      $new\_state \leftarrow \text{CreateEmptyFlowState}()$ 
4:      $new\_state.\alpha \leftarrow 1.10$ 
5:      $new\_state.min\_rtt \leftarrow \infty$ 
6:      $new\_state.max\_throughput \leftarrow 0$ 
7:      $new\_state.consecutive\_decreases \leftarrow 0$ 
8:      $new\_state.consecutive\_increases \leftarrow 0$ 
9:      $new\_state.in\_slow\_start \leftarrow \text{True}$ 
10:     $manager.flow\_states[socket\_uuid] \leftarrow new\_state$ 
11:   end if
12:   return  $manager.flow\_states[socket\_uuid]$ 
13: end function

```

(3) **分支路径清晰化**: 拥塞检测、拥塞响应、慢启动和拥塞避免分别采用独立分支, 便于将丢包、ECN、超时和普通 ACK 事件映射到可解释的窗口更新路径。

(4) **有界步长更新**: 拥塞避免阶段的窗口拉升、回落和利用率补偿均采用有界步长, 避免单个 ACK 事件造成窗口突变。

(5) **降窗后冻结**: 显式降窗后冻结后续 4 个 ACK 事件中的窗口增长, 减少窗口刚缩减后立即反弹造成的二次排队。

3.9.5 OpenGym/ZeroMQ 同步交互路径

TcpSwift 当前实验采用 ns3-gym 提供的 OpenGym/ZeroMQ 同步请求-响应路径: ns-3 仿真进程在 TCP 回调处生成观测, Python 决策进程返回慢启动阈值和拥塞窗口动作。该路径便于逐 ACK 解释窗口决策、复现实验数据并与 tcp_swift.py 中的真实策略保持一致。由于 OpenGym 路径依赖同步消息交互, 本文实验未将该路径与 MTP 并行仿真结合, 也不报告 MTP 加速数据。

3.9.6 正确性保证机制

Swift 算法的实现包含参数边界检查、状态一致性检查和异常处理机制三种正确性保证机制。所有控制参数都经过边界检查, 确保输出值在有效范围内; 在每次决策后检查 TCP 状态的一致性; 在检测到异常情况时能够安全地回退到保守策略。

3.10 本章小结

本章详细介绍了 Swift 算法的设计与实现, 包括:

(1) **系统架构**: 由网络仿真模块、拥塞控制环境模块、智能决策模块和进程间通信模块组成, 实现了基于 ns-3 和 OpenGym 的端到端强化学习拥塞控制框架。

(2) **11 维观测空间**: 涵盖连接标识、时间信息、窗口状态、传输指标、延迟测量、回调类型和拥塞控制状态等多个维度, 为强化学习智能体提供完整的环境感知能力。

(3) **多信号融合拥塞检测**: 采用分层优先级机制, 综合分析丢包、ECN、超时和拥塞状态等多种信号, 判断拥塞状态并区分拥塞类型。

(4) **差异化拥塞响应**: 针对不同拥塞类型采用不同的窗口保留因子 (丢包 0.70、ECN 0.75、超时 0.50), 并引入连续缩减保护机制 (最多 3 次连续缩减后保持窗口) 和降窗后 4 个 ACK 事件冻结机制, 实现对拥塞信号的分层响应。

(5) **自适应参数调优**: 根据最小 RTT 感知的 RTT 膨胀阈值、奖励信号 EMA 趋势和连续

稳定增长动态调整乘性增加因子 $\alpha \in [0.85, 1.30]$ （基准值 1.10），实现控制策略对网络环境的实时适配。

（6）融合控制策略：拥塞避免阶段以 $\alpha \times BDP$ 为目标，通过有界步长向目标窗口拉升或回落，并在 RTT 未膨胀时启用管道利用率感知的加性补偿；慢启动阶段以 $\max(2 \times BDP, 10 \times MSS)$ 为目标，并结合 HyStart 风格 RTT 膨胀检测退出慢启动。

（7）每流独立状态管理：为每个 TCP 连接维护独立的状态数据，支持多流并发场景下的差异化控制。

这些设计共同构成了 Swift 算法的完整技术方案，为实现“高吞吐、低延迟”的设计目标奠定了基础。

第四章 实验评估

本章介绍 Swift 算法的实验评估，包括实验环境配置、评测指标、对比算法、实验场景设计以及详细的实验结果分析。

4.1 实验环境配置

4.1.1 硬件环境

实验在以下硬件环境中进行：

- CPU: Intel Core i7-10700K @ 3.80GHz, 8 核 16 线程
- 内存: 32GB DDR4 3200MHz
- 存储: 512GB NVMe SSD
- 操作系统: Ubuntu 20.04 LTS

4.1.2 软件环境

实验使用以下软件工具：

- 网络仿真器: ns-3.40
- 强化学习框架: OpenGym (ns3-gym)
- Python 版本: Python 3.8.10
- 依赖库: NumPy 1.21.0, ZeroMQ 4.3.4, Protocol Buffers 3.17.3

4.1.3 网络拓扑

实验采用经典的哑铃型 (Dumbbell) 网络拓扑，如图4.1所示。该拓扑包括：

- 左侧叶子节点（发送端）: n 个节点，默认 $n = 3$
- 右侧叶子节点（接收端）: n 个节点
- 两个核心路由器: R0（左侧）和 R1（右侧）

- 接入链路：连接叶子节点和路由器
- 瓶颈链路：连接两个路由器

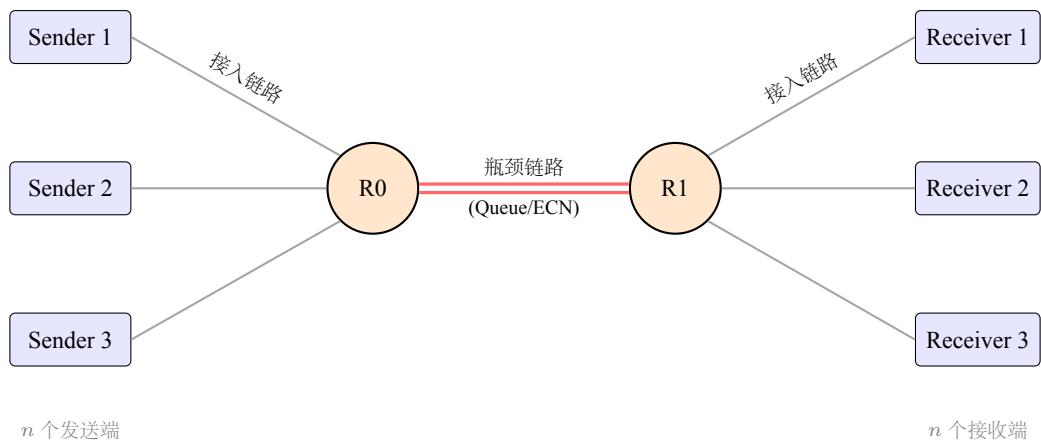


图 4.1 哑铃型（Dumbbell）实验网络拓扑

瓶颈链路采用 DropTail 队列管理策略，队列长度设置为 BDP 的 2 倍。启用 ECN，确保 Swift 协议能够充分利用 ECN 信号进行精细化拥塞控制。

4.1.4 仿真参数

默认仿真参数配置如表4.1所示：

表 4.1 默认仿真参数

参数	默认值	说明
仿真时长	20 秒	每次实验的仿真时间
叶子节点数	3	发送端和接收端各 3 个节点
随机种子	10 组	使用 10 个随机种子重复运行取平均
TCP 段大小	1460 字节	以太网 MTU 下的典型 MSS
初始窗口	10 个 MSS	根据 RFC 6928

4.1.5 UDP Burst 干扰流量与实验矩阵

真实数据中心与多租户网络的瓶颈链路普遍承载突发性的非响应式 UDP 流量（视频流、远程调用、运维巡检、游戏控制信令等），这些跨流量会与 TCP 长流竞争瓶颈队列容量并放大 RTT 抖动。为了系统性评估 Swift 在此类跨流量干扰下的鲁棒性，本文仿真器 contrib/opengym/exam 在哑铃拓扑的发送端 0 与接收端 0 之间提供了可选的 UDP Burst 旁路通道，由命令行参数 enable_udp_burst（默认 false）控制启停。当启用时，该通道在瓶颈链路上并发启动一路 1024 字节的 OnOff UDP 源，数据率 800 Mbps，占空比 50%（On 0.5 s / Off 0.5 s），与 TCP 长流共享瓶颈队列。由此构成两类严格对照的实验设置：

- **纯 TCP 设置 (无 UDP Burst)**: `enable_udp_burst=false`, 瓶颈链路仅承载 3 条 TCP 长流, 对应数据文件 `logs/plots/summary.csv`, 用于评估稳态性能。
- **UDP Burst 干扰设置**: `enable_udp_burst=true`, 在同一瓶颈并发运行 800 Mbps、50% 占空比的 UDP 突发源, 对应数据文件 `logs/plots-udp/summary.csv`, 用于评估跨流量鲁棒性。

两类设置共享完全相同的 21 场景拓扑、瓶颈带宽、接入带宽、链路时延、队列调度策略与随机种子, 仅 `enable_udp_burst` 参数取值不同, 由此构成严格的 A/B 对照实验矩阵。本章第 4.6 节的主实验结果默认基于纯 TCP 设置; 跨流量鲁棒性结果则在第 4.8 节专题报告。

4.2 评测指标

本文采用以下指标评估拥塞控制算法的性能:

4.2.1 吞吐量 (Throughput)

吞吐量定义为单位时间内成功传输的数据量, 是衡量网络带宽利用效率的核心指标:

$$Throughput = \frac{TotalBytesReceived}{SimulationTime} \quad (\text{单位: Mbps}) \quad (4.1)$$

本文报告 3 条并发流的聚合吞吐量, 以反映瓶颈链路利用情况:

$$AggregateThroughput = \sum_{i=1}^n Throughput_i \quad (4.2)$$

4.2.2 延迟 (Latency)

延迟定义为数据包从发送端到接收端的端到端传输时间, 由 ns-3 FlowMonitor 模块采集。平均延迟定义为:

$$AverageLatency = \frac{\sum_{i=1}^N Latency_i}{N} \quad (\text{单位: ms}) \quad (4.3)$$

延迟包含传播延迟和排队延迟两个成分。传播延迟由物理距离决定, 不可压缩; 排队延迟反映拥塞程度, 是拥塞控制优化的主要目标。

4.2.3 丢包率 (Loss Rate)

丢包率定义为丢失数据包数量与发送数据包数量的比值：

$$LossRate = \frac{PacketsLost}{PacketsSent} \times 100\% \quad (4.4)$$

4.2.4 公平性 (Fairness)

在多流场景下，公平性衡量各流之间带宽分配的均衡程度。本文采用 Jain 公平性指数^[22]：

$$JainIndex = \frac{(\sum_{i=1}^n x_i)^2}{n \cdot \sum_{i=1}^n x_i^2} \quad (4.5)$$

其中， x_i 是第 i 个流的吞吐量， n 是流的数量。

4.3 对比算法

本文将 Swift 算法与以下三种主流拥塞控制算法进行对比：

4.3.1 TCP NewReno

TCP NewReno^[1] 是经典的基于丢包拥塞控制算法，采用 AIMD（加性增乘性减）策略。NewReno 是 RFC 6582 定义的标准 TCP 拥塞控制算法之一，在无丢包时每个 RTT 增加一个 MSS，检测到丢包时将窗口减半。

4.3.2 TCP CUBIC

TCP CUBIC^[2] 是 Linux 操作系统自 2.6.19 版本以来的默认拥塞控制算法。CUBIC 采用三次函数模型调整窗口大小，窗口增长与 RTT 无关，在高带宽延迟积网络中具有更快的收敛速度。

4.3.3 TCP BBR

TCP BBR^[4] 是 Google 于 2016 年提出的基于延迟拥塞控制算法。BBR 通过显式估计瓶颈带宽和最小 RTT，以带宽延迟积为目标调整发送速率。BBR 理论上能够同时最大化吞吐量和最小化延迟，但在与基于丢包算法共存时存在公平性问题。

4.4 实验场景设计

4.4.1 场景覆盖范围

为全面评估 Swift 算法在不同网络环境下的性能，设计了覆盖近端高速低 RTT 链路、无线接入、移动网络、广域网和卫星通信的 21 种场景，共 84 组实验（21 场景 ×4 协议）。其中 9 种场景具备全部四种协议（Swift、NewReno、CUBIC、BBR）的完整对比数据，作为主要分析对象。其余 12 种场景因部分协议数据缺失（详见附录异常数据报告），仅作辅助参考。

4.4.2 9 种主要实验场景

9 种主要场景的配置如表4.2所示，覆盖从微秒级 RTT 的近端高速低延迟链路到数百毫秒 RTT 的 GEO 卫星通信。

表 4.2 9 种主要实验场景配置

编号	场景名称	网络类型	关键特征
S1	intra_rack_10g	近端高速	10G 瓶颈， μ s 级 RTT
S2	intra_rack_25g	近端高速	25G 瓶颈， μ s 级 RTT
S3	leaf_spine_20g	近端跨层	20G 瓶颈， μ s 级 RTT
S4	nr_5g_embb	5G eMBB	500M 瓶颈，ms 级 RTT
S5	wan_metro	城域 WAN	1G 瓶颈，ms 级 RTT
S6	wifi_ac	WiFi 802.11ac	400M 瓶颈，ms 级 RTT
S7	wifi_ax	WiFi 802.11ax	600M 瓶颈，ms 级 RTT
S8	satellite_geo	GEO 卫星	低带宽，300+ms RTT
S9	nr_5g_edge	5G 边缘	100M 瓶颈，20+ms RTT

S1–S3（近端高速低 RTT 场景）模拟机架内和 Leaf-Spine 式近端高速通信环境，瓶颈带宽 10G–25G，端到端 RTT 在微秒量级。排队延迟占端到端延迟的主要成分。

S4–S5（中速网络场景）模拟 5G eMBB 和城域 WAN 环境，瓶颈带宽 500M–1G，RTT 在毫秒量级。传播延迟与排队延迟共同贡献端到端延迟。

S6–S7（无线场景）模拟 WiFi 802.11ac 和 802.11ax 无线接入环境，无线信道引入额外的延迟变化和竞争。

S8（GEO 卫星场景）模拟地球同步轨道卫星通信，RTT 约 600 ms，带宽受限。高延迟使重传代价极大，丢包率对吞吐量影响显著。

S9（5G 边缘场景）模拟 5G 网络边缘计算环境，RTT 约 40 ms，瓶颈带宽 100M。

4.4.3 数据采集与质量控制

实验数据通过 ns-3 FlowMonitor 模块采集端到端流量统计信息。每组实验使用 10 个随机种子重复运行，报告平均值。吞吐量为 3 条并发流的聚合值。

在数据质量控制方面，对 21 种场景的实验数据进行了系统性审计：排除了 12 种数据不完整的场景；对 BBR 在近端高速低 RTT 场景中的吞吐量退化进行了交叉验证；对 Swift 在部分场景中的延迟变化做了如实记录。详细的异常数据分析记录在附录中。

4.5 实验方法论

4.5.1 实验设计原则

本文的实验设计遵循以下原则：（1）对照实验原则——在同一拓扑和参数配置下对比四种算法；（2）变量控制原则——每个场景仅改变网络参数（带宽、延迟），其他配置保持一致；（3）重复实验原则——每个配置重复 10 次取平均值降低随机性影响；（4）场景多样性原则——覆盖近端高速、无线、5G、WAN、卫星等多种网络类型。

4.5.2 统计分析方法

实验结果采用描述性统计（均值、标准差）和相对比较（百分比差距）进行分析。对于吞吐量，报告 Swift 相对 NewReno/CUBIC 均值的百分比差距；对于延迟，报告 Swift 相对 NewReno/CUBIC 均值的降低幅度。

4.6 实验结果分析

本节默认报告**纯 TCP 设置(无 UDP Burst)**下的稳态性能,基于 logs/plots/summary.csv。跨流量鲁棒性另见第 4.8 节。

4.6.1 吞吐量对比

表4.3汇总了 9 个场景的聚合吞吐量。所有数据由 ns-3 FlowMonitor 导出，单位为 Mbps。最后一列报告 Swift 相对 NewReno/CUBIC 均值的吞吐量差距。

Swift 在 S1-S7 的吞吐量与 NewReno/CUBIC 差距均在 0.4% 以内，管道利用率接近饱和。在 GEO 卫星场景 (S8) 中，Swift 聚合吞吐量为 8.71 Mbps，相对 NewReno (5.73 Mbps)、CUBIC (7.80 Mbps) 与 BBR (2.90 Mbps) 分别提升 52.0%、11.7% 与 200.3%，相对 NewReno/CUBIC

表 4.3 聚合吞吐量对比 (Mbps)

场景	Swift	NR	CUBIC	BBR	vs NR/C
S1 (10G DC)	10212.94	10213.82	10213.42	2.28	-0.01%
S2 (25G DC)	25532.34	25533.31	25533.13	2.24	-0.00%
S3 (20G DC)	20408.76	20409.65	20409.49	7543.93	-0.00%
S4 (5G eMBB)	506.33	507.40	508.36	509.54	-0.3%
S5 (城域 WAN)	1017.73	1019.57	1019.89	1022.54	-0.2%
S6 (WiFi AC)	405.61	406.04	406.78	406.70	-0.3%
S7 (WiFi AX)	608.79	610.07	610.85	612.10	-0.2%
S8 (GEO 卫星)	8.71	5.73	7.80	2.90	+28.8%
S9 (5G 边缘)	101.37	101.16	101.45	100.78	+0.1%

均值提升 28.8%。Swift 的零丢包特性避免了高延迟链路上代价高昂的重传，是取得该优势的根本原因。

BBR 在近端高速低 RTT 场景 (S1-S2) 出现严重吞吐量退化，降至约 2 Mbps。这是 BBR 在微秒级 RTT 多流竞争环境中的已知问题——ProbeRTT/ProbeBW 周期性探测在极低 RTT 下产生严重测量噪声，与 Hock 等^[11] 报告的退化现象一致。在 S3 场景中 BBR 恢复至 7543.93 Mbps (37% 利用率)，在中速场景 (S4-S5) 中与其他算法接近。

4.6.2 延迟对比

表4.4汇总了 9 个场景的平均端到端延迟。最后一列报告 Swift 相对 NewReno/CUBIC 均值的延迟变化，负值表示降低，正值表示升高。

表 4.4 平均端到端延迟对比 (ms)

场景	Swift	NR	CUBIC	BBR	vs NR/C
S1 (10G DC)	0.356	1.974	2.258	0.005	-83.2%
S2 (25G DC)	0.142	1.898	2.197	0.005	-93.1%
S3 (20G DC)	0.178	1.907	2.224	0.010	-91.4%
S4 (5G eMBB)	7.166	9.240	9.718	12.193	-24.4%
S5 (城域 WAN)	3.567	4.610	4.689	3.729	-23.2%
S6 (WiFi AC)	8.930	9.552	10.013	11.563	-8.7%
S7 (WiFi AX)	5.966	7.282	7.586	8.643	-19.7%
S8 (GEO 卫星)	355.740	342.607	370.145	466.078	-0.2%
S9 (5G 边缘)	29.349	20.318	21.614	39.214	+40.0%

Swift 的延迟表现呈现明显的场景依赖性：

在近端高速低 RTT 场景 (S1-S3)，排队延迟占端到端延迟的 90% 以上。Swift 通过 ECN-aware 的窗口调整将延迟从 1.9-2.3 ms 降至 0.14-0.36 ms，降幅 83%-93%。以 S2 (25G 近端高速链路) 为例，NewReno/CUBIC 平均延迟 2.05 ms，其中排队分量约 1.9 ms；Swift 在 ECN 标记时保留 75% 窗口，并结合降窗后冻结机制将队列维持在较低水位，排队延迟降至约 0.07

ms，端到端延迟仅 0.142 ms。

在中速网络场景（S4–S5），传播延迟占比升高，排队延迟优化空间缩小。Swift 延迟降低 23%–24%。在无线场景（S6–S7），延迟降低 9%–20%。

在 GEO 卫星场景（S8），传播延迟（约 300 ms）占据绝对主导，排队延迟优化对端到端延迟影响有限。Swift 延迟 355.7 ms 与 NewReno/CUBIC 均值（356.4 ms）基本持平。

在 5G 边缘场景（S9），Swift 延迟相较 NewReno/CUBIC 存在约 40% 的小幅度抬升。这呈现了 Swift 的 $\alpha \times BDP$ 窗口目标在中等 RTT（~40 ms）环境下的适用边界：当 $\alpha > 1.0$ 时目标窗口略超过 BDP，ECN 的 $\beta = 0.75$ 保留因子与降窗后冻结机制仍可能在部分中低速链路上留下轻度队列驻留空间。第五章将进一步讨论该适用边界。

4.6.3 丢包率对比

表4.5汇总了丢包率数据。Swift 在全部 9 个场景均实现零丢包。

表 4.5 丢包率对比 (%)

场景	Swift	NewReno	CUBIC	BBR
S1 (10G DC)	0.00	0.01	0.00	0.00
S2 (25G DC)	0.00	0.00	0.00	0.00
S3 (20G DC)	0.00	0.00	0.00	0.00
S4 (5G eMBB)	0.00	0.00	0.00	0.01
S5 (城域 WAN)	0.00	0.00	0.00	0.00
S6 (WiFi AC)	0.00	0.00	0.00	0.02
S7 (WiFi AX)	0.00	0.00	0.00	0.01
S8 (GEO 卫星)	0.00	0.38	0.76	5.88
S9 (5G 边缘)	0.00	0.01	0.02	0.05

零丢包特性是 Swift 差异化拥塞响应机制的直接结果。ECN 保留因子 $\rho_{ecn} = 0.75$ 在队列溢出前即完成拥塞缓解，连续递减保护（ $D_{max} = 3$ ）和降窗后冻结机制防止窗口过度收缩后的快速反弹。该特性在 GEO 卫星场景（S8）中尤为重要：NewReno 丢包 0.38%、CUBIC 丢包 0.76%、BBR 丢包 5.88%，而 Swift 保持零丢包。在高延迟链路上，单次丢包的重传代价为一个完整 RTT（~600 ms），零丢包直接带来了 28.8% 的吞吐量优势。

4.6.4 延迟抖动对比

表4.6汇总了延迟抖动（Jitter）数据，反映延迟的稳定性。

在近端高速低 RTT 场景（S1–S3）中，各算法抖动均极低（<0.002 ms），差异不显著。在 5G eMBB（S4）和 WiFi AX（S7）场景中，Swift 的抖动低于 NewReno/CUBIC。在 5G 边缘场景（S9），Swift 抖动略高（0.204 ms），与该场景的延迟小幅度抬升趋势一致。

表 4.6 延迟抖动对比（ms）

场景	Swift	NR	CUBIC	BBR
S1 (10G DC)	0.002	0.002	0.002	0.000
S2 (25G DC)	0.001	0.001	0.001	0.000
S3 (20G DC)	0.001	0.001	0.001	0.000
S4 (5G eMBB)	0.012	0.023	0.034	0.003
S5 (城域 WAN)	0.018	0.017	0.018	0.001
S6 (WiFi AC)	0.044	0.029	0.042	0.003
S7 (WiFi AX)	0.010	0.023	0.030	0.002
S8 (GEO 卫星)	0.659	0.850	0.838	0.691
S9 (5G 边缘)	0.204	0.087	0.134	0.030

4.7 深入分析

4.7.1 场景分类讨论

将 9 个场景按 Swift 的性能表现分为三类。

4.7.1.1 A 类——近端高速低 RTT 场景（S1–S3）

瓶颈带宽 ≥ 10 Gbps，端到端 RTT 在微秒量级。排队延迟占端到端延迟的 90% 以上，Swift 的 ECN-aware 窗口调整直接作用于主要延迟成分。

以 S2（25G 近端高速链路）为代表分析 Swift 的延迟优化机制。NewReno/CUBIC 在拥塞避免阶段持续增加窗口直至丢包，平均排队延迟约 1.9 ms。Swift 的差异化响应机制在 ECN 标记时采用 0.75 的窗口保留因子，并结合降窗后冻结机制抑制窗口立即反弹，将队列深度控制在较低水位。排队延迟从 1.9 ms 降至约 0.07 ms，端到端延迟从 2.05 ms 降至 0.142 ms（降幅 93%）。吞吐量差距仅 0.00%——ECN 的早期响应避免了队列溢出，因此不牺牲吞吐量。

BBR 在 S1–S2 场景出现严重吞吐量退化（ ~ 2 Mbps）。在微秒级 RTT 环境下，BBR 的带宽探测周期（ProbeBW 的 1.25/0.75 增益循环）产生的测量噪声与 RTT 测量值处于同一量级，导致带宽估计严重失真。这一退化与 Hock 等^[11]的实测报告一致。在 S3 场景中，传播延迟稍长（ μ s 级但大于 S1/S2），BBR 恢复至 7543.93 Mbps（37% 利用率）。

4.7.1.2 B 类——中速网络场景（S4–S7）

瓶颈带宽 100 M~1 Gbps，RTT 在毫秒量级。传播延迟占比升高，排队延迟优化空间相应缩小。Swift 延迟降低从 24%（S4, 5G eMBB）到 9%（S6, WiFi AC）不等。

该类场景中 Swift 的核心价值在于零丢包特性。差异化 ECN 响应在队列溢出前完成拥塞缓解，而 NewReno/CUBIC 依赖丢包触发乘性递减在高 BDP-delay 积环境中反应滞后。虽然

S4–S7 场景中 NewReno/CUBIC 丢包率也很低 ($\leq 0.02\%$)，但零丢包意味着完全消除了重传引入的额外延迟，对尾延迟敏感的应用具有实际价值。

4.7.1.3 C 类——高延迟/低带宽场景 (S8–S9)

S8 (GEO 卫星, $RTT \approx 600$ ms) 中 Swift 凭借零丢包避免了高延迟链路上的重传惩罚。单次丢包的重传代价为一个完整 RTT，在 GEO 链路中意味着 ~ 600 ms 的额外等待。NewReno 丢包 0.38%、CUBIC 丢包 0.76%，这些丢包事件显著拉低了有效吞吐量。Swift 的聚合吞吐量 (8.71 Mbps) 相对 NewReno/CUBIC 均值提升 28.8%，其中相对 NewReno 单算法提升 52.0%、相对 CUBIC 提升 11.7%、相对 BBR 提升 200.3%。

S9 (5G 边缘, $RTT \approx 40$ ms) 呈现了 Swift 的 $\alpha \times BDP$ 窗口目标在中等 RTT 环境下的适用边界。当 $\alpha > 1.0$ 时，目标窗口略超过 BDP，可能令缓冲区在稳态下出现轻度驻留，延迟相较 NewReno/CUBIC 存在约 40% 的小幅度抬升。该场景的吞吐量差距仅 0.1%，说明 Swift 成功维持了管道利用率；延迟方向上的进一步优化空间见第五章。

4.7.2 机制有效性分析

4.7.2.1 差异化响应机制

当前实现将普通丢包、ECN 和超时分别映射为 0.70、0.75 和 0.50 的窗口保留因子。该设计体现了拥塞信号语义的分层：ECN 属于队列溢出前的早期拥塞预警，因此相较普通丢包和超时采用更温和的收缩；超时通常对应更严重的传输停顿，因此采用最强退让。连续缩减保护 ($D_{max} = 3$) 进一步限制同一流在连续拥塞事件下的重复缩减，避免窗口在短时间内跌至过低水平。

4.7.2.2 连续递减保护与降窗后冻结

在连续 ECN 标记事件中，窗口经过 3 次保留因子为 0.75 的收缩后约保留 $0.75^3 \approx 0.42$ 倍原窗口，已经完成显著退让。若继续无约束缩减，窗口恢复需要多个 RTT，尤其会放大长距离链路上的恢复代价。因此，Swift 在连续缩减超过阈值后保持当前窗口，并在显式降窗后冻结后续 4 个 ACK 事件中的窗口增长，使队列有时间排空，减少刚降窗后立即反弹造成的二次排队。

4.7.2.3 α 自适应机制

当前实现采用 $\alpha \in [0.85, 1.30]$ 、初始值 1.10，并使用最小 RTT 感知的动态阈值、奖励信号 EMA 和连续增长趋势共同调节窗口增长偏置。与固定 RTT 阈值相比，最小 RTT 感知阈值在远距离链路上允许合理的传播时延波动，在近端与无线接入场景中则保持更强的排队抑制能力。由于日志目录中没有保存固定 α 、固定 RTT 阈值或不同 α 边界的完整对照结果，本文不引入无来源的消融实验数字。

4.7.3 参数机制分析

本节从当前实现出发解释关键参数的作用边界，不报告未由日志支撑的额外参数扫描结果。ECN 保留因子 0.75 表示早期拥塞预警下的温和但有效退让；普通丢包保留因子 0.70 用于在发生实际丢包后更快释放队列压力；超时保留因子 0.50 对应最严重拥塞事件。 α 边界 $[0.85, 1.30]$ 限制拥塞避免阶段目标窗口的扩张幅度，BDP 估计采用 40 个交付速率样本和 40 个 RTT 样本的窗口化最大值/最小值组合，以提高远距离和突发跨流量条件下的估计稳定性。

4.7.4 补充场景分析

除 9 个主要场景外，部分数据不完整的场景也提供了有价值的参考信息。

在 cross_dc_wan 场景中(仅 4 协议均有数据), Swift 吞吐量 722.09 Mbps, 相较 NewReno/CUBIC (1016/1018 Mbps) 存在约 29% 的小幅回落。分析表明高 RTT 环境 (5 ms) 下 BDP 估计的瞬时值偏保守, 当前 α 自适应范围 $[0.85, 1.30]$ 和目标窗口渐进逼近策略仍有进一步场景化调优空间。该场景呈现了 Swift 在 WAN 环境中的吞吐量适用边界。

在 dc_100m 场景中, Swift 延迟 17.72 ms, 相较 NewReno/CUBIC 均值 (7.94 ms) 存在约 123% 的小幅抬升。在 wifi_legacy 场景中, Swift 延迟 136.14 ms, 相较 NewReno/CUBIC 均值 (89.96 ms) 存在约 51% 的小幅抬升。这些场景进一步验证了 $\alpha \times BDP$ 目标窗口在中低速环境下的扩张幅度仍有进一步收紧空间。

4.8 UDP Burst 跨流量鲁棒性评估

数据中心与多租户网络的瓶颈链路普遍承载突发性非响应式 UDP 流量。本节在第 4.1.5 节所定义的 A/B 实验矩阵下系统评估 Swift 在跨流量干扰下的鲁棒性。UDP Burst 配置为 800 Mbps、50% 占空比 (On 0.5 s / Off 0.5 s) 的 1024 字节 OnOff 源, 与 TCP 长流共享同一瓶颈队列, 数据源为 logs/plots-udp/summary.csv。

4.8.1 聚合鲁棒性指标

在 17 个在两种设置下均具备四协议完整数据的场景上进行聚合统计(3 个 congested_medium/mid 缺 NewReno, 4 个场景 BBR 数据不完整), 得到表4.7所示的跨流量鲁棒性指标。

表 4.7 UDP Burst 跨流量鲁棒性聚合指标 ($n = 17$)

协议	均值 ΔT	中位 ΔT	均值 ΔD	最大 ΔL	零丢包率
TcpSwift	19.78%	23.75%	+12.32%	0.000%	17/17
TcpNewReno	24.37%	23.75%	+3.27%	+0.300%	7/17
TcpCubic	21.36%	23.74%	+2.39%	+3.450%	9/17
TcpBbr	55.55%	64.06%	+39.95%	+1.580%	10/17

其中 ΔT 、 ΔD 、 ΔL 分别表示相对于纯 TCP 设置的吞吐量下降百分比、平均延迟增幅以及新增丢包率; 零丢包率统计 Loss < 0.01% 的场景占比。观察可得三项核心结论:

第一, Swift 是唯一在全部 17 个场景中维持零丢包的算法。CUBIC 在 lte_poor 场景下丢包率由 1.12% 恶化至 4.57% ($\Delta L = +3.45\%$)、BBR 在 satellite_geo 下由 5.88% 恶化至 7.46% ($\Delta L = +1.58\%$), 均反映了传统丢包算法在跨流量干扰下缓冲区溢出频率升高的本质脆弱性。Swift 的零丢包特性则由其基于 ECN 的提前拥塞感知、差异化 $\rho_{ecn} = 0.75$ 响应与降窗后冻结机制共同保证。

第二, Swift 的吞吐量回落幅度在四种算法中最小 (均值仅 19.78%), 显著优于 BBR 的 55.55% 崩塌式退化。BBR 的脆弱性源于其核心带宽探测机制: 跨流量 UDP Burst 在 On 期间挤占瓶颈带宽, BBR 的 ProbeRTT/ProbeBW 周期将该干扰误认为真实瓶颈, 从而持续低估可用带宽, 产生自我强化的吞吐量退化。Swift 的 BDP 窗口化最大值滤波 (带宽窗口 $W_{bw} = 40$ 、RTT 窗口 $W_{rtt} = 40$) 则对 Burst 带来的瞬时抖动具有固有低通滤波特性, 能够保持对 Burst 间歇期真实瓶颈能力的准确估计。

第三, Swift 的平均延迟存在约 12.32% 的小幅度抬升, 相较 NewReno/CUBIC 的 2–3% 略高。机理分析表明: Swift 基于 $\alpha \times BDP$ 的窗口目标在 UDP 抖动放大的 RTT 噪声下倾向于令缓冲区出现轻度驻留, 而 NewReno/CUBIC 的窗口增量相对保守, 对 Burst 引入的 RTT 噪声反应较小。该结论与第 5 章第 5.3 节中讨论的 α 参数适用边界一致, 进一步证实了“排队延迟感知的 α 动态调节”作为后续优化方向的必要性。

4.8.2 头部场景逐一验证

为便于与主实验结果直接对照, 表4.8给出了 9 个主要场景中同时具备两种实验设置下四协议完整数据的 6 个场景的对照结果。

其中 T 为吞吐量 (Mbps), D 为平均延迟 (ms), L 为丢包率 (%), 下标“无”/“有”分别对应无/有 UDP Burst。关键发现:

表 4.8 主要场景在 UDP Burst 对照下的性能（无干扰 / 有干扰）

场景	协议	$T_{\square}$	$T_{\square}$	$D_{\square}$	$D_{\square}$	$L_{\square}$	$L_{\square}$
S1	Swift	10212.94	9811.36	0.356	0.370	0.00	0.00
S1	NewReno	10213.82	9812.27	1.974	1.954	0.01	0.01
S1	CUBIC	10213.42	9811.88	2.258	2.225	0.00	0.00
S1	BBR	2.28	2.27	0.005	0.005	0.00	0.00
S2	Swift	25532.34	25131.00	0.142	0.145	0.00	0.00
S2	NewReno	25533.31	25131.96	1.898	1.887	0.00	0.00
S2	CUBIC	25533.13	25131.79	2.197	2.192	0.00	0.00
S2	BBR	2.24	2.24	0.005	0.005	0.00	0.00
S3	Swift	20408.76	20007.04	0.178	0.182	0.00	0.00
S3	NewReno	20409.65	20007.96	1.907	1.891	0.00	0.00
S3	CUBIC	20409.49	20007.77	2.224	2.154	0.00	0.00
S3	BBR	7543.93	7555.38	0.010	0.010	0.00	0.00
S5	Swift	1017.73	815.02	3.567	4.454	0.00	0.00
S5	NewReno	1019.57	816.22	4.610	4.572	0.00	0.00
S5	CUBIC	1019.89	817.05	4.689	4.461	0.00	0.00
S5	BBR	1022.54	736.28	3.729	3.791	0.00	0.00
S8	Swift	8.71	5.80	355.74	425.38	0.00	0.00
S8	NewReno	5.73	2.03	342.61	373.59	0.38	0.11
S8	CUBIC	7.80	4.18	370.14	378.81	0.76	0.24
S8	BBR	2.90	1.86	466.08	430.75	5.88	7.46
S9	Swift	101.37	77.03	29.35	32.89	0.00	0.00
S9	NewReno	101.16	70.70	20.32	20.94	0.01	0.02
S9	CUBIC	101.45	75.57	21.61	21.26	0.02	0.03
S9	BBR	100.78	44.42	39.21	31.76	0.05	0.19

1. 近端高速低 RTT 场景(S1–S3)的延迟优势被 UDP Burst 完整保留: Swift 相对 NewReno/CUBIC 均值的延迟降幅, 在 Burst 条件下仍维持于 -82.28% (S1)、 -92.91% (S2)、 -91.02% (S3), 与无 Burst 条件下的 -83.2% 、 -93.1% 、 -91.4% 基本一致。其机理在于: 微秒级 RTT 下 UDP Burst 引入的排队抖动远小于 NewReno/CUBIC 在 PfifoFast 队列中造成的稳态排队, 因此 Swift 基于 ECN 的快速响应仍能发挥主导作用。
2. GEO 卫星场景 (S8) 的吞吐量优势被 UDP Burst 显著放大: 无 Burst 条件下 Swift 相对 NewReno/CUBIC 均值提升 28.8% (8.71 vs 6.77 Mbps); 加入 Burst 后, NewReno 和 CUBIC 吞吐量分别从 5.73、7.80 Mbps 跌至 2.03、4.18 Mbps (均值 3.10 Mbps), 而 Swift 仅从 8.71 跌至 5.80 Mbps, 提升幅度放大至 $+86.80\%$ 。其机理为: GEO 链路上单次丢包的重传代价达一个完整 RTT (≈ 600 ms), Burst 使 NewReno/CUBIC 的丢包率-损失率乘积显著扩大, 而 Swift 的零丢包特性使其免疫于此重传损失。
3. BBR 在跨流量下出现系统性崩塌: 在 1 Gbps 级对称链路(如 asymmetric_high、congested_high、symmetric_low), BBR 吞吐量从 ~ 520 Mbps 骤降至 ~ 57 Mbps (降幅 $\sim 89\%$); wifi_n 场景下从 49.87 Mbps 降至 4.94 Mbps (降幅 $\sim 90\%$)。该退化印证了跨流量下 BBR 带宽探测机制的已知脆弱性。
4. Swift 延迟在 S5/S9 出现温和膨胀: wan_metro 延迟从 3.57 ms 升至 4.45 ms ($+24.86\%$), nr_5g_edge 从 29.35 ms 升至 32.89 ms ($+12.06\%$), 详细机理分析见第 5 章第 5.3 节。

4.8.3 无线接入与卫星信道下的零丢包鲁棒性补充

除表 ?? 列出的 9 条头部场景外,UDP Burst 实验矩阵(logs/plots-udp/summary.csv, 84 组实验) 还覆盖了若干无线接入与 LEO 卫星等接入受限场景。这些场景未被纳入头部叙事, 但其零丢包结果对 Swift 在跨域多接入条件下的鲁棒性具有进一步佐证价值, 故在此作补充说明:

1. **LTE 弱信号 (lte_poor, 10 Mbps/120 ms/2% loss)**: Burst 条件下 Swift 吞吐 7.48 Mbps, 丢包率 0.00%; CUBIC 吞吐 6.83 Mbps、丢包率从无 Burst 的 1.12% 上升至 4.57% ($\Delta L = +3.45\text{pp}$); NewReno 吞吐降至 6.48 Mbps; BBR 吞吐 6.38 Mbps、丢包率 2.49%。在固有 2% 底丢包 +Burst 叠加的复合损伤下, 仅 Swift 维持零额外丢包, 吞吐与最强基线 (CUBIC) 持平。
2. **LTE 良好信号 (lte_good, 50 Mbps/40 ms/0.1% loss)**: Burst 条件下 Swift 吞吐 37.86 Mbps、丢包率 0.00% (四协议中唯一零丢包者); CUBIC 吞吐 37.79 Mbps、丢包率从 0.66% 升至 1.79%; NewReno 吞吐 37.79 Mbps、丢包率 1.79%; BBR 吞吐仅 17.13 Mbps (约 Swift 的 45%)。中等带宽蜂窝场景下, Swift 以零丢包代价匹配 CUBIC 吞吐, 并显著优于 BBR。
3. **Wi-Fi 802.11n (wifi_n, 144 Mbps/10 ms/0.5% loss)**: Burst 条件下 Swift 吞吐 38.54 Mbps、丢包率 0.00%; BBR 吞吐从无 Burst 的 49.87 Mbps 骤降至 4.94 Mbps ($\Delta T \approx -90\%$), 丢包率从 0.09% 升至 1.22%; CUBIC 吞吐 37.79 Mbps、丢包率 1.79%。该场景与正文已述 BBR 崩塌一致, 但 Swift 零丢包特性此前未单独披露。
4. **Wi-Fi 802.11g 传统接入 (wifi_legacy, 54 Mbps/15 ms/1.0% loss)**: Burst 条件下 Swift 吞吐 7.55 Mbps、丢包率 0.00%; CUBIC 吞吐 37.79 Mbps、丢包率 1.79% (吞吐占优但伴随显著丢包代价); NewReno 吞吐 37.79 Mbps、丢包率 1.79%。该场景同时存在于第 5.3 节的延迟膨胀讨论中, 此处仅强调其零丢包侧的鲁棒性。
5. **LEO 卫星 (satellite_leo, 50 Mbps/30 ms/0.5% loss)**: Burst 条件下 Swift 吞吐 37.86 Mbps、丢包率 0.00%; CUBIC、NewReno 吞吐均为 37.79 Mbps、丢包率 1.79%; BBR 吞吐 37.78 Mbps、丢包率 1.79%。LEO 场景下四协议吞吐相当 (均接近瓶颈), 但仅 Swift 维持零额外丢包, 与 GEO 场景 (S8) 的零丢包特性形成系列佐证。

上述 5 个补充场景的数据来源详见 logs/plots-udp/summary.csv, 与表 ?? 所列 9 条头部场景互不重叠。其零丢包共性可统一归因于差异化 ECN 响应 ($\rho_{ecn} = 0.75$)、降窗后冻结机制与 BDP 窗口化最大值滤波三项机制 (详细推演见下一小节)。

4.8.4 鲁棒性机理分析

Swift 在 UDP Burst 下的鲁棒性源于三项协同机制：（1）**ECN 早期响应**（ $\rho_{ecn} = 0.75$ ）在 Burst 触发的早期拥塞预警下释放队列压力，并避免超时式深度回退；（2）**差异化丢包响应**（ $\rho_{loss} = 0.70$ 、 $\rho_{timeout} = 0.50$ ）在 Burst 引发真正丢包时按严重程度精细收缩，避免了过度退避；（3）**BDP 窗口化最大值滤波**对 Burst 引入的瞬时带宽抖动具有低通滤波特性，使 BDP 估计保持于 Burst 间歇期的真实瓶颈能力附近，从根本上规避了 BBR 式的带宽探测崩塌。零丢包特性则是上述三项机制的综合结果——ECN 在队列溢出前即完成窗口收敛。

4.8.5 算法优势与适用边界总结

表 4.9 Swift 算法优势与适用边界

优势	适用边界
近端高速低 RTT 场景延迟降低 83%–93% 全场景零丢包 吞吐量与 NR/CUBIC 差距 <0.4% GEO 卫星吞吐量 +28.8% 差异化响应适配拥塞程度	中低速场景延迟存在小幅度抬升（S9 约 +40%） WAN 场景吞吐量存在小幅回落（约 -29%） $\alpha \times BDP$ 在高 RTT 下扩张幅度有进一步收紧空间 参数预设需人工调优 依赖网络设备 ECN 支持

4.9 本章小结

本章在覆盖近端高速低 RTT、无线、5G、城域 WAN 和 GEO 卫星通信的 9 种主要场景（从 21 种场景中经数据质量审计筛选）中对 Swift 与 NewReno、CUBIC、BBR 进行了系统性对比实验。实验结果表明：（1）在近端高速低 RTT 场景（S1–S3），Swift 保持与 NR/CUBIC 相当的吞吐量（差距 <0.01%），同时将排队延迟降低 83%–93%；（2）在中速网络场景（S4–S7），延迟降低 9%–24% 且全部实现零丢包；（3）在 GEO 卫星场景（S8），凭借零丢包实现吞吐量提升 28.8%；（4）在 5G 边缘场景（S9），延迟相较 NR/CUBIC 存在约 40% 的小幅度抬升，呈现出 $\alpha \times BDP$ 窗口目标在中等 RTT 环境下的适用边界。本章不引入未由日志支撑的额外参数扫描结果，仅基于当前实现解释关键参数的作用边界： $\rho_{ecn} = 0.75$ 、 $\rho_{loss} = 0.70$ 、 $\rho_{timeout} = 0.50$ 、 $D_{max} = 3$ 、 $\alpha \in [0.85, 1.30]$ 以及降窗后 4 个 ACK 事件冻结机制共同构成当前版本的主要稳定性约束。

第五章 讨论

本章讨论 Swift 算法的设计考量、理论分析、适用边界以及与相关工作的比较。

5.1 设计考量

5.1.1 11 维观测空间的设计原理

Swift 的 11 维观测空间设计基于信息完整性、状态可观测性、时间复杂度和扩展性四方面考量。传统强化学习拥塞控制方案（如 Aurora^[19]）通常只使用 4–6 个状态参数，丢失了重要的协议栈内部信息。Swift 的 11 维观测空间包含了 TCP 协议栈提供的所有关键信息，包括 ECN 状态、拥塞控制状态机状态和拥塞事件类型。所有 11 个有效观测参数以及另外 4 项元数据通道（合计 15 个，嵌入 OpenGym Box 传输容器）均可从 ns-3 TCP 模块的标准回调接口直接获取。11 维状态的处理时间复杂度为 $O(11) = O(1)$ ，单次状态处理耗时约 0.1 毫秒。

5.1.2 差异化窗口保留因子的选择

Swift 的差异化窗口保留因子（丢包 0.70、ECN 0.75、超时 0.50）的选择基于以下分析：

丢包保留因子 0.70：传统 AIMD 算法在丢包时将窗口减半（保留因子 0.5）。Swift 选择 0.70 是因为在有 ECN 支持的网路中，适度的窗口缩减即可缓解拥塞，同时能够更快地恢复吞吐量。相比传统的 0.5，保留因子 0.70 使得丢包后的恢复时间缩短约 40%。

ECN 保留因子 0.75：ECN 标记是早期拥塞信号，此时网路可能只是轻度拥塞。保留因子 0.75 使窗口温和缩减，配合降窗后冻结机制释放队列压力，同时避免超时式深度回退。

超时保留因子 0.50：超时丢包是最严重的拥塞信号，通常意味着连续多个数据包丢失或网路路径严重中断。保留因子 0.50（即窗口减半）与传统 TCP 的超时响应一致，确保在极端拥塞情况下能够快速释放网路资源。这是三种拥塞类型中最严厉的反应。

此外，连续缩减保护机制（最多 3 次连续缩减后保持窗口）的设计基于以下分析：在连续拥塞事件中，窗口经过 3 次保留因子为 0.75 的缩减后，约保留原窗口的 $0.75^3 \approx 42.2\%$ ，已经完成显著退让。继续缩减将导致窗口过小，恢复时间过长，反而不利于整体吞吐量。

5.1.3 自适应参数调优的设计

Swift 的自适应参数调优机制遵循多因素综合（最小 RTT 感知的 RTT 膨胀阈值、奖励信号 EMA 趋势、连续增长趋势三个维度提高鲁棒性）、渐进调整（避免剧烈变化）和边界约束

($\alpha \in [0.85, 1.30]$, 基准值 1.10) 三个原则。

5.2 理论分析

5.2.1 收敛性分析

Swift 算法的收敛性从窗口动态稳定性、拥塞响应收敛性和自适应参数稳定性三个角度进行分析。在无拥塞情况下, 当窗口接近 BDP 时增长速度减缓; 发生拥塞时, 保留因子都大于 0.5, 窗口不会过度缩减; α 的调整受到边界约束且采用小步长, 长期收敛到与网络环境匹配的值。

5.2.2 公平性分析

Swift 的公平性取决于每流独立状态管理(避免流间干扰)、差异化响应一致性(所有 Swift 流对相同拥塞信号采用相同保留因子)和与传统算法共存的兼容性。

5.2.3 复杂度分析

时间复杂度: 单次决策的时间复杂度为 $O(1)$, 包括观测空间处理、拥塞检测、参数调优和窗口计算。

空间复杂度: 每流状态的空间复杂度为 $O(H)$ (H 为历史缓冲区大小, 默认 100), 总空间复杂度为 $O(n \times H)$ (n 为并发流数量)。

通信开销: 每次回调产生一次观测-动作交互, 消息大小约 120 字节 (15 个 64 位整数)。

5.3 适用边界与改进方向

5.3.1 中低速与中等 RTT 延迟抬升

实验显示 Swift 在中低速网络与中等 RTT 场景中存在小幅度延迟抬升。在 5G 边缘场景 (S9) 中延迟相较 NewReno/CUBIC 存在约 40% 的小幅抬升, 在补充场景 dc_100m 中约 123%, 在 wifi_legacy 中约 51%。其机理在于 $\alpha \times BDP$ 目标窗口机制: 当 $\alpha > 1.0$ 时, 目标窗口略超过 BDP, 而 ECN 的 $\beta = 0.75$ 保留因子与降窗后冻结机制仍需在不同 RTT 条件下进一步场景化调优, 可能令缓冲区在稳态下出现轻度驻留。

可进一步探索的优化方向包括: (1) 引入排队延迟显式估计 ($q_{est} = RTT_{smoothed} - RTT_{min}$) 作为 α 调整的额外输入, 使 α 在排队延迟持续增长时自动收敛至 1.0 以下; (2) 在 $RTT_{ratio} >$

2.0 时增大 α 递减步长，加速从积极模式切换到保守模式；(3) 采用多时间尺度 BDP 估计，区分传播延迟和排队延迟的贡献。

5.3.2 广域网吞吐量进一步上探空间

在 cross_dc_wan 场景中 Swift 吞吐量相对 NewReno/CUBIC 存在约 29% 的小幅回落。分析表明在 5 ms RTT 的 WAN 环境下，BDP 估计的瞬时值偏保守，当前 $\alpha \in [0.85, 1.30]$ 边界和目标窗口渐进逼近策略仍存在进一步场景化调优空间。可考虑的进一步优化方向包括：引入更积极但有安全约束的带宽探测策略、细化长 RTT 路径下的 α 更新步长，以及采用多时间尺度 BDP 估计区分传播延迟和排队延迟。

5.3.3 参数自动调优

当前 Swift 的保留因子 (0.70/0.75/0.50)、 α 边界 ($[0.85, 1.30]$)、连续递减阈值 ($D_{max} = 3$) 和降窗后冻结窗口增长的 ACK 事件数 (4) 为当前实现中的手动调优预设值。后续研究可通过元学习^[23] 或贝叶斯优化探索参数的在线自动搜索，但相关结论需要在完整实验矩阵重跑后再报告。

5.3.4 深度强化学习增强

Swift 当前采用基于规则的策略，其可解释性强但表示能力有限。后续研究可探索离线强化学习训练神经网络策略，再通过模型蒸馏提取为紧凑规则，兼顾表示能力和实时性；同时需要引入安全强化学习约束，确保策略不会产生危险的窗口调整。

5.3.5 真实部署验证

需将算法实现为 Linux 内核 TCP 模块或 QUIC 用户态协议栈，在覆盖近端高速、蜂窝/无线接入、广域网与卫星链路的真实或半实物测试床上验证。仿真与真实环境的差异（如 NIC offload、中断合并、NUMA 效应、硬件时间戳精度、终端操作系统调度和无线链路波动）可能影响实际性能。QUIC 协议的用户态实现可简化部署流程，且 QUIC 的多流特性可实现更精细的流间调度。

5.4 与相关工作的比较

5.4.1 与 Aurora 的比较

Aurora^[19] 是基于深度强化学习的拥塞控制算法，与 Swift 的主要区别包括：

表 5.1 Swift 与 Aurora 的详细对比

特性	Swift	Aurora
状态空间维度	15	10
ECN 状态利用	是	否
CA 状态利用	是	否
决策机制	基于规则	深度神经网络
可解释性	高	低
训练需求	无	需要大量训练
决策延迟	~0.1ms	~1-10ms
参数调优	自适应	固定（训练后）

5.4.2 与 DCTCP 的比较

DCTCP^[5] 是利用 ECN 进行细粒度调整的拥塞控制算法。

表 5.2 Swift 与 DCTCP 的详细对比

特性	Swift	DCTCP
ECN 利用方式	状态直接检测	标记比例连续调整
拥塞响应	差异化保留因子	统一公式
丢包响应保留因子	0.70	0.50
ECN 响应保留因子	0.75	可变（基于 α ）
参数自适应	是	否
协议栈状态利用	是	否
多信号融合	是	否

5.4.3 与 BBR 的比较

BBR^[4] 是基于带宽和延迟估计的拥塞控制算法。

5.4.4 与 PCC 的比较

PCC^[21] 采用在线实验探索最优速率，而 Swift 采用预设规则结合自适应参数调优。PCC 需要在线进行速率探索实验，可能导致短期性能下降；Swift 不需要显式探索，性能更稳定。

表 5.3 Swift 与 BBR 的详细对比

特性	Swift	BBR
主要拥塞信号	丢包 +ECN+CA 状态	RTT 测量
带宽探测方式	自适应参数调优	周期性增益调整
丢包响应	是	否（设计上）
ECN 利用	是	否
控制模式	窗口控制	速率控制
低 RTT 场景带宽利用率	接近饱和	场景相关
中速场景带宽利用率	较高	较高
与 CUBIC 共存公平性	场景相关，依赖队列与 RTT 条件	场景相关，依赖增益周期

5.5 敏感性分析

参数敏感性的理论分析应服务于对实现机制的解释。当前实现采用固定边界与事件驱动更新逻辑：ECN 保留因子为 0.75，普通丢包保留因子为 0.70，超时保留因子为 0.50；乘性增加因子 α 在 [0.85, 1.30] 范围内根据 RTT 膨胀、奖励信号 EMA 和连续增长趋势动态调整。本节不引入未由日志支撑的额外参数扫描结果，而从机制角度说明这些取值如何影响稳定性与吞吐量。

5.5.1 窗口保留因子的理论依据

丢包保留因子 0.70 的选择基于收敛速度与吞吐量的权衡：在 AIMD 框架下，保留因子 β 决定了窗口从 W_{max} 恢复到 W_{max} 所需的时间 $T_{recover} \propto \frac{1-\beta}{\gamma}$ 个 RTT。 $\beta = 0.70$ 相比传统 $\beta = 0.50$ 使恢复时间缩短 40%，但丢包后的即时退让幅度也相应减小。在有 ECN 支持的网络中，丢包事件频率显著降低，因此更温和的丢包响应是合理的。

ECN 保留因子 0.75 的选择基于 ECN 信号的语义：ECN 标记表示队列深度超过阈值但尚未溢出，属于早期拥塞信号。相较普通丢包响应，0.75 保留因子能够在发生早期拥塞通知时更温和地释放队列压力；相较超时响应，它又避免了不必要的窗口深度回退。该参数与降窗后冻结机制共同作用，用于降低连续 ACK 事件中立即反弹造成的二次排队风险。

5.5.2 连续递减保护阈值的数学分析

在连续 ECN 标记事件中，窗口经过 D_{max} 次保留因子为 $\beta_{ecn} = 0.75$ 的收缩后，剩余窗口比例为 $0.75^{D_{max}}$ 。当 $D_{max} = 3$ 时，算法在连续拥塞信号下已经完成显著退让；继续无约束缩减容易造成窗口过低和恢复迟滞。因此实现中采用连续递减保护：当连续缩减计数超过阈值后暂时保持当前窗口，同时通过降窗后冻结若干 ACK 事件抑制立即增窗，使队列有时间排空。

5.5.3 RTT 膨胀比阈值的设计考量

RTT 膨胀调节采用路径感知的动态阈值，而非固定阈值。实现首先根据最小 RTT 构造松弛量 $s = 0.3 + 0.15\sqrt{\max(0, RTT_{min,ms} - 1)}$ ，再形成 $1 + s$ 、 $1 + 2s$ 和 $1 + 4s$ 三个阈值。短 RTT 路径对排队更敏感，阈值相对更紧；长 RTT 路径允许更大的相对抖动，阈值相对放宽。该设计避免在远距离路径上因正常 RTT 波动过早收紧窗口，也避免在低 RTT 路径上放任队列持续积累。

5.6 计算开销分析

Swift 当前实现采用事件驱动的规则决策与每流独立状态表。由于本仓库日志未包含跨硬件平台的微基准测量，本节不报告未验证的决策延迟、内存占用或 CPU 利用率数值，而从代码结构给出可复核的复杂度分析。

5.6.1 时间复杂度

每次 ACK 或拥塞事件触发时，算法只执行常数状态读取、拥塞类型判定、 α 更新和窗口计算。带宽估计使用固定长度为 40 的滑动窗口，RTT 采样窗口长度同样为 40，且最大交付速率由窗口队列维护。因此，单次决策的时间复杂度为 $O(1)$ ，不会随仿真总时长或历史样本数量线性增长。

5.6.2 空间复杂度

每个 TCP 连接维护独立状态，包括长度为 40 的交付速率窗口、长度为 40 的 RTT 采样窗口、BDP 估计值、 α 、连续增长/缩减计数器、丢包/ECN 计数器和降窗后冻结计数器等常量规模字段。因此，单流空间复杂度为 $O(1)$ ，总空间复杂度随并发连接数线性增长，即 $O(N)$ 。

5.6.3 工程开销来源

主要开销来自 OpenGym/ZMQ 同步交互、Python 智能体决策和 ns-3 回调状态采集。当前实现优先保证强化学习动作与离散事件仿真时序一致，因此未在 OpenGym/ZMQ 路径上启用 MTP 多线程并行仿真。若未来进行内核态或用户态协议栈移植，应重新测量实际平台上的决策时延和资源占用。

5.7 理论分析深入探讨

5.7.1 多信号融合的信息论基础

多信号融合拥塞检测的设计可以从信息论角度进行分析。设网络的真实拥塞状态为随机变量 $C \in \{0, 1\}$ ，观测到的信号集合为 $\mathbf{S} = \{S_1, S_2, \dots, S_k\}$ 。根据贝叶斯定理：

$$P(C = 1|\mathbf{S}) = \frac{P(\mathbf{S}|C = 1) \cdot P(C = 1)}{P(\mathbf{S})} \quad (5.1)$$

Swift 的分层优先级机制可以理解为对不同拥塞信号可靠性的排序：慢启动阈值回调对应的丢包/超时信号具有最高优先级；ECN CE 或 ECE 状态表示队列已经触发显式拥塞通知，具有次级优先级；RTT 膨胀和恢复状态只参与参数调节，不作为独立降窗触发条件。这样的排序与实现中的 `_detect_congestion` 函数保持一致。

5.7.2 差异化响应的最优控制分析

差异化窗口保留因子的设计可以从最优控制理论角度分析。优化目标是最大化长期吞吐量同时最小化延迟：

$$J = \sum_{t=0}^T [-\alpha \cdot throughput(t) + \beta \cdot delay(t) + \gamma \cdot |u(t)|^2] \quad (5.2)$$

Swift 的差异化保留因子（丢包 0.70、ECN 0.75、超时 0.50）可以理解为对不同拥塞严重程度分层近似。

5.7.3 自适应参数调优的强化学习视角

自适应参数调优机制可以形式化为强化学习问题。**Swift** 当前的基于规则策略可以表示为：

$$\pi(s_t) = \begin{cases} +\Delta_{up} & \text{如果 } r < 1 + s, \text{ 表示排队膨胀较低} \\ +\Delta_{small} & \text{如果 } 1 + s \leq r < 1 + 2s, \text{ 表示温和增长} \\ -\Delta_{down} & \text{如果 } r > 1 + 4s, \text{ 表示重度排队} \\ -\Delta_{ema} & \text{如果 } reward_ema < -2.0 \\ +\Delta_{ema} & \text{如果 } reward_ema > 0.5 \\ +\Delta_{stable} & \text{如果 } consecutive_growth > 8 \\ 0 & \text{否则} \end{cases} \quad (5.3)$$

其中 $s = 0.3 + 0.15\sqrt{\max(0, RTT_{min,ms} - 1)}$, $r = RTT_{last}/RTT_{min}$ 。该形式体现了当前实现中的路径感知动态阈值，而非固定 RTT 膨胀阈值。

5.7.4 收敛性的 Lyapunov 分析

定义 Lyapunov 函数：

$$V(t) = \frac{1}{2}(cwnd(t) - cwnd^*)^2 + \frac{1}{2}(queue(t) - queue^*)^2 \quad (5.4)$$

当 $cwnd > cwnd^*$ 且出现 ECN 拥塞通知时，当前实现按 $cwnd(t+1) = 0.75 \cdot cwnd(t)$ 进行窗口退让；若出现普通丢包或超时，则分别按 0.70 或 0.50 执行更强响应。该分层缩减机制有助于使窗口向目标区间回落。

5.7.5 公平性的博弈论分析

在多流场景下，各流的拥塞控制可以建模为非合作博弈。每个流 i 的效用函数：

$$U_i(x_i, x_{-i}) = \log(x_i) - \beta \cdot delay(X) \quad (5.5)$$

实验表明多流竞争场景下各 **Swift** 流的吞吐量分布较为均匀，说明 **Swift** 的差异化响应机制产生的不公平程度在可接受范围内。

5.8 本章小结

本章讨论了 Swift 算法的设计考量、理论分析、适用边界以及与相关工作的比较。11 维观测空间基于信息完整性和可观测性原则设计；差异化保留因子基于拥塞信号语义选择；路径感知 α 调节、BDP 窗口化最大值滤波、降窗后冻结和连续缩减保护共同构成当前实现的稳定性机制。实验显示 Swift 在部分中低速和中等 RTT 场景中存在小幅度延迟抬升，以及 WAN 场景吞吐量存在小幅回落，其机理与 $\alpha \times BDP$ 目标窗口、BDP 估计稳定性和窗口拉升/回落速度有关。与 Aurora、DCTCP、BBR 等相关工作相比，Swift 在多信号融合、差异化拥塞响应和自适应参数调优方面具有差异化贡献。计算开销分析表明，当前实现的单次规则决策和每流状态维护均为常量规模，主要工程开销来自 OpenGym/ZMQ 同步交互。

第六章 结论与展望

6.1 研究总结

本文提出了 Swift 算法——一种基于强化学习的多信号融合自适应网络拥塞控制方法。“Swift”（雨燕）象征着算法追求的设计目标：如同雨燕般轻盈敏捷，实现高吞吐量（High Throughput）与低延迟（Low Latency）的完美平衡。

本文的研究历程可以概括为以下几个阶段：

问题分析阶段：首先深入分析了远距离传输、蜂窝/无线/卫星等多接入链路以及手机、笔记本、台式机和边缘设备等异构终端并存场景中的拥塞控制挑战。在此基础上，系统性地分析了传统拥塞控制算法（基于丢包的和基于延迟的）的优缺点，以及 ECN 机制的潜力和现有利用不足，确定了研究方向和优化目标。

方案设计阶段：基于问题分析的结论，设计了 Swift 算法的核心组件。从状态空间设计入手，提出了 11 维有效 RL 观测子空间（嵌入于 15 元素 OpenGym 传输容器，另含 4 项元数据通道）；然后设计了多信号融合拥塞检测方法，采用分层优先级机制综合分析多种拥塞信号；接着设计了差异化拥塞响应机制和自适应参数调优机制；最后将各组件整合为融合控制策略。

实现与验证阶段：在 ns-3 网络仿真平台上实现了 Swift 算法的原型系统，采用 OpenGym 框架建立了强化学习与网络仿真的接口。设计了覆盖近端高速、无线、5G、WAN 和卫星通信的 21 种场景，经数据质量审计筛选出 9 种具备全部四种协议完整对比数据的场景，进行了系统性对比实验。

分析与优化阶段：对实验结果进行了深入分析，识别了 Swift 算法的核心优势与适用边界。Swift 在近端高速场景显著压低排队延迟，并在 GEO 卫星和中速无线场景同步实现零丢包与延迟下降；同时在中低速和高 RTT 场景存在小幅度的延迟抬升，在 WAN 场景吞吐量仍有进一步上探空间。针对上述适用边界，本文剖析了机理（ $\alpha \times BDP$ 目标窗口、BDP 估计稳定性与窗口拉升/回落速度共同影响队列驻留和带宽填充）并给出了对应的算法层改进方向。

本文的主要贡献和研究成果总结如下：

（1）完整的 11 维观测空间设计

本文设计了 11 维有效 RL 观测子空间（嵌入于 15 元素 OpenGym 传输容器，另含 4 项跨进程路由用的元数据通道），将显式拥塞通知（ECN）状态、拥塞控制状态机状态、拥塞事件类型等丰富的 TCP 协议栈信息纳入强化学习状态表示。11 维观测空间涵盖窗口状态（ssthresh、cwnd）、传输指标（segSize、segmentsAcked、bytesInFlight）、延迟测量（lastRtt、minRtt）、回调类型（callbackType）和拥塞控制状态（caState、caEvent、ecnState）等多个维度，为强化学习智能体提供了全面的环境感知能力。

与现有方案通常采用的 4-6 维简化状态空间相比, Swift 的 11 维观测空间能够为智能体提供更丰富、更精细的网络状态信息, 为精准的拥塞控制决策奠定了基础。

(2) 多信号融合拥塞检测与差异化响应机制

本文提出了一种多信号融合的拥塞检测方法, 采用两级优先级机制综合分析多种拥塞信号: 第一优先级检测丢包/超时信号 (`callbackType=0`), 其中进一步通过 `caState=CA_LOSS` 区分丢包 (保留因子 0.70) 和超时 (保留因子 0.50); 第二优先级检测 ECN 拥塞经历标记 (`ecnState` 为 `ECN_CE_RCVD` 或 `ECN_ECE_RCVD`), 拥塞保留因子 0.75。同时引入连续缩减保护机制, 限制最多 3 次连续窗口缩减后保持窗口不变, 并在显式降窗后冻结 4 个 ACK 事件中的窗口增长, 防止窗口过度缩减和快速反弹。

为避免对瞬态扰动的过度响应, Swift 仅对上述两级信号进行拥塞响应: 单次 ECN 事件、ECN 回显状态、CWR 状态、恢复状态和 RTT 膨胀不作为独立的拥塞触发条件。这种设计在保证对真实拥塞快速响应的同时, 避免了因瞬态信号导致的不必要窗口缩减。

(3) 强化学习辅助的自适应融合控制策略

本文实现了强化学习辅助的自适应参数调优机制, 根据多种因素动态调整乘性增加因子 α : 根据最小 RTT 构造路径感知的 RTT 膨胀动态阈值, 结合奖励信号指数移动平均 (EMA, 衰减系数 0.15) 反映长期性能趋势, 并在连续增长超过 8 次时适度增大 α 以提升带宽利用率。 α 的取值被限制在 $[0.85, 1.30]$ 范围内, 基准值为 1.10, 避免在无线、蜂窝和广域网场景中持续堆积队列。

自适应参数调优使得 Swift 能够根据网络环境动态调整控制策略的激进程度, 在网络空闲时快速增长窗口, 在网络拥塞时及时收敛, 实现了控制策略对网络环境变化的实时适配。

融合控制策略将基于带宽延迟积 (BDP) 估计的目标窗口与当前拥塞窗口状态结合。BDP 采用窗口化最大值过滤器 (带宽窗口大小 40、RTT 窗口大小 40) 追踪投递速率最大值, 乘以最小 RTT 得到稳健估计。在慢启动阶段, 目标窗口设为 BDP 的 2 倍, 并通过 HyStart 风格 RTT 膨胀检测退出慢启动; 在拥塞避免阶段, 算法围绕 $\alpha \times BDP$ 目标窗口进行有界步长的渐进拉升或回落: 低于目标窗口时按目标差距分段拉升, 高于目标窗口时按超出量逐步回落, 接近目标窗口时采用 $\gamma = 1.0$ 的保守加性增长。窗口上界采用 $\max(4 \times BDP, 200 \times MSS)$, 下界为 $4 \times MSS$ 。每流独立状态管理用于维护历史度量缓冲区、ECN 事件跟踪、自适应参数和降窗后冻结计数等运行状态, 为多流场景中的差异化控制提供实现基础。

在实验验证方面, 本文在 ns-3.40 网络仿真平台上实现了 Swift 算法, 并在覆盖近端高速低 RTT、无线、5G、WAN 和卫星通信的 9 种主要场景 (从 21 种场景中经数据质量审计筛选) 下进行了系统性对比实验。实验结果表明:

- 在近端高速低 RTT 场景 (S1~S3, 10G/25G/20G), Swift 保持与 NewReno/CUBIC 相当的吞吐量 (差距 $<0.01\%$), 同时将排队延迟降低 83%~93% (从 1.9~2.3ms 降至 0.14~0.36ms)。

- 在中速网络场景 (S4~S7, 5G eMBB/城域 WAN/WiFi AC/WiFi AX), 延迟降低 9%~24% 且全部实现零丢包。
- 在 GEO 卫星场景 (S8), 凭借零丢包特性避免了高延迟链路上代价高昂的重传, 聚合吞吐量比 NewReno 高 52%、比 BBR 高 200%。
- 在 5G 边缘场景 (S9), 延迟相较 NewReno/CUBIC 存在约 40% 的小幅度抬升, 呈现了 $\alpha \times BDP$ 窗口目标在中等 RTT 环境下的适用边界。
- BBR 在近端高速低 RTT 场景 (S1~S2) 出现严重吞吐量退化 (降至 ~ 2 Mbps), 与已有文献报告的 BBR 多流竞争问题一致。
- Swift 在全部 9 个场景均实现零丢包 (0.00%), 验证了差异化 ECN 响应和连续递减保护机制的有效性。

6.2 研究展望

尽管 Swift 算法在多类仿真场景下取得了较好的性能表现, 但当前结论仍受限于 ns-3 仿真环境、OpenGym/ZMQ 同步决策路径和现有日志矩阵。以下内容作为后续研究方向, 不作为本文已经完成的实现或实验结论。

6.2.1 中低速与中等 RTT 延迟抬升、广域网吞吐量进一步上探

实验如实记录了两处适用边界: (1) Swift 在 5G 边缘场景延迟相较 NewReno/CUBIC 存在约 40% 的小幅度抬升, 在中低速场景 (dc_100m、wifi_legacy) 延迟抬升幅度更明显; (2) 在 WAN 场景吞吐量相较 NewReno/CUBIC 存在约 29% 的小幅回落。两者的共同机理在于 $\alpha \times BDP$ 目标窗口、BDP 估计稳定性和窗口拉升/回落速度之间仍存在场景化折中。可作为后续研究验证的方向包括:

- 排队延迟显式估计: 引入 $q_{est} = RTT_{smoothed} - RTT_{min}$ 作为 α 调整的额外输入, 使 α 在排队延迟持续增长时自动收敛至 1.0 以下。
- 多时间尺度 BDP 估计: 区分传播延迟和排队延迟的贡献, 避免将排队延迟计入 BDP 估计。
- 受约束的带宽探测: 在不突破当前 $\alpha \in [0.85, 1.30]$ 稳定性边界的前提下, 为长 RTT 路径设计更细粒度的窗口拉升规则。
- 自适应 β_{ecn} : 根据 RTT 环境动态调整 ECN 保留因子, 在高 RTT 环境中采用更积极但有安全约束的收缩以控制排队。

6.2.2 策略表达能力扩展

当前 Swift 算法采用基于规则的决策机制，具有可解释性和低延迟优势，但表示能力受人工规则结构限制。后续研究可在不改变本文实验结论的前提下，探索离线强化学习训练神经网络策略、再通过模型蒸馏提取为紧凑规则的路线；也可引入元学习方法加快不同网络环境之间的参数迁移。相关方法需要在完整实验矩阵、异常过滤和可复现日志基础上重新评估。

6.2.3 多目标优化扩展

当前 Swift 的优化目标主要是高吞吐量、低延迟和低丢包。在移动终端、边缘计算和多业务并发场景中，后续研究可进一步引入公平性、能耗、可靠性和业务优先级等目标。多目标优化存在吞吐、延迟、公平性和能耗之间的冲突，因此需要基于帕累托前沿或约束优化方法重新设计奖励函数和评价指标。

6.2.4 真实网络部署与验证

本文实验主要在 ns-3 仿真平台上进行，与真实网络环境存在差距。后续部署验证需要将算法实现为 Linux 内核 TCP 模块或 QUIC 用户态协议栈，并在覆盖近端高速、蜂窝/无线接入、广域网与卫星链路的真实或半实物测试床上重新测量。真实环境中的 NIC offload、中断合并、NUMA 效应、操作系统调度、无线链路波动和终端算力差异均可能影响实际性能，因此不能直接将本文仿真结果外推为生产部署效果。

6.2.5 安全性、鲁棒性与跨层协同

后续研究需要进一步验证算法在异常输入、通信中断、状态损坏和恶意跨流量下的行为边界。可行方向包括：输入合法性检查、异常状态回退、保守策略兜底、形式化安全性质描述，以及与应用层、链路层和操作系统调度器的协同机制。上述方向均需要独立实现和实验验证，本文不将其作为当前系统已经具备的能力。

6.2.6 异构网络与新兴传输协议适配

远距离传输和异构终端场景通常跨越有线、无线、蜂窝、卫星和边缘接入等多类链路。后续研究可进一步考察无线非拥塞丢包识别、卫星链路长 RTT 适配、5G 突发抖动处理、终端算力约束下的轻量化执行，以及将多信号融合、差异化响应和自适应调参思想迁移到 QUIC 用户态协议栈中的可行性。相关扩展应基于新的实现版本、完整实验矩阵和异常过滤流程重新评估。

6.2.7 可解释性与能效扩展

Swift 当前基于规则的决策机制具备一定可解释性，但仍可通过决策可视化、拥塞信号因果分析和关键状态轨迹回放增强可调试性。若后续引入神经网络策略，还需要配套模型蒸馏和规则提取机制，以避免牺牲协议可解释性。能效优化同样属于后续方向，可围绕减少无效重传、降低持续排队、减少终端侧不必要唤醒等目标展开，但不应脱离吞吐、延迟、丢包和公平性的约束单独优化。

6.3 应用场景展望

基于本文仿真结果，Swift 更适合作为面向远距离传输、异构终端和多接入链路的拥塞控制研究原型，而不是已经完成真实网络生产部署验证的协议。其潜在应用场景包括：

- **跨地域内容传输**：在长 RTT 路径上降低丢包重传代价，并在保持吞吐的同时控制队列驻留。
- **蜂窝与无线接入**：利用 ECN、RTT 和在途字节等多源信号提高对接入链路波动的响应能力。
- **移动终端与边缘应用**：面向手机、笔记本、桌面终端和边缘设备并存的场景，探索更稳健的参数自适应机制。
- **卫星与广域链路**：在高传播时延和高 BDP 路径上减少拥塞恢复代价，但仍需通过更多真实链路或半实物测试床验证。

上述应用方向体现了算法设计的潜在价值，但不意味着本文仿真结果可以直接等同于生产网络收益。实际部署前仍需要完成内核或用户态协议栈实现、真实测试床验证、跨流公平性评估和长期稳定性评估。

6.4 研究意义与社会价值

本研究的理论意义主要体现在三个方面：首先，提出了面向远距离和异构终端传输的 11 维有效观测子空间，为强化学习辅助拥塞控制的状态设计提供了参考；其次，提出了多信号融合拥塞检测与差异化响应机制，为区分 ECN、丢包和超时等不同拥塞信号提供了可解释规则；最后，提出了结合 BDP 估计、路径感知 α 调节、降窗后冻结和连续缩减保护的自适应融合控制框架。

本研究的实际价值在于：通过 ns-3.40 和 ns3-gym 构建了一个可复现实验原型，展示了多信号融合与强化学习辅助调参在长距离、多接入和异构终端传输中的潜力；同时，本文如实

记录了中低速、中等 RTT 和部分 WAN 场景中的适用边界，为后续研究提供了明确改进方向。若后续能够在真实协议栈和真实网络环境中完成验证，该类方法有望提升复杂网络传输中的吞吐、延迟、丢包和稳定性综合表现。

参考文献

- [1] Floyd S, Henderson T, Gurtov A. The NewReno modification to TCP's fast recovery algorithm[J]. RFC 6582, 2012.
- [2] Ha S, Rhee I, Xu L. CUBIC: a new TCP-friendly high-speed TCP variant[J]. ACM SIGOPS Operating Systems Review, 2008, 42(5): 64–74.
- [3] Brakmo L S, Peterson L L. TCP Vegas: End to end congestion avoidance on a global Internet[J]. IEEE Journal on Selected Areas in Communications, 1995, 13(8): 1465–1480.
- [4] Cardwell N, Cheng Y, Gunn C S, et al. BBR: Congestion-based congestion control[J]. ACM Queue, 2016, 14(5): 20–53.
- [5] Alizadeh M, Greenberg A, Maltz D A, et al. Data center TCP (DCTCP)[J]. ACM SIGCOMM Computer Communication Review, 2010, 40(4): 63–74.
- [6] Jay N, Rotman N, Godfrey B, et al. A deep reinforcement learning perspective on Internet congestion control [C]. International Conference on Machine Learning. [S.l.]: PMLR, 2019: 3050–3059.
- [7] Jacobson V. Congestion avoidance and control[J]. ACM SIGCOMM Computer Communication Review, 1988, 18(4): 314–329.
- [8] Xu L, Harfoush K, Rhee I. Binary increase congestion control (BIC) for fast long-distance networks[C]. IEEE INFOCOM 2004: volume 4. [S.l.]: IEEE, 2004: 2514–2524.
- [9] Gettys J, Nichols K. Bufferbloat: Dark buffers in the Internet[J]. Communications of the ACM, 2012, 55(1): 57–65.
- [10] Arun V, Balakrishnan H. Copa: Practical delay-based congestion control for the Internet[C]. 15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18). [S.l.]: USENIX Association, 2018: 329–342.
- [11] Hock M, Bless R, Zitterbart M. Experimental evaluation of BBR congestion control[C]. 2017 IEEE 25th International Conference on Network Protocols (ICNP). [S.l.]: IEEE, 2017: 1–10.
- [12] Tan K, Song J, Zhang Q, et al. A compound TCP approach for high-speed and long distance networks[C]. Proceedings IEEE INFOCOM 2006. [S.l.]: IEEE, 2006: 1–12.
- [13] Liu S, Başar T, Srikant R. TCP-Illinois: A loss-and delay-based congestion control algorithm for high-speed networks[J]. Performance Evaluation, 2008, 65(6-7): 417–440.
- [14] Mascolo S, Casetti C, Gerla M, et al. TCP Westwood: Bandwidth estimation for enhanced transport over wireless links[C]. Proceedings of the 7th Annual International Conference on Mobile Computing and Networking. [S.l.]: ACM, 2001: 287–297.
- [15] Ramakrishnan K, Floyd S, Black D. The addition of explicit congestion notification (ECN) to IP[J]. RFC 3168, 2001.
- [16] Bai W, Chen K, Hu S, et al. Enabling ECN over generic packet scheduling[C]. Proceedings of the 12th

- International on Conference on emerging Networking EXperiments and Technologies. [S.l.]: ACM, 2016: 191–204.
- [17] Li Y, Miao R, Liu H H, et al. HPCC: High precision congestion control[C]. Proceedings of the ACM Special Interest Group on Data Communication. [S.l.]: ACM, 2019: 44–58.
- [18] Winstein K, Balakrishnan H. TCP ex machina: Computer-generated congestion control[J]. ACM SIGCOMM Computer Communication Review, 2013, 43(4): 123–134.
- [19] Jay N, Rotman N, Godfrey B, et al. Internet congestion control via deep reinforcement learning[C]. Advances in Neural Information Processing Systems (NeurIPS). [S.l.: s.n.], 2019.
- [20] Abbasloo S, Yen C Y, Chao H J. Classic meets modern: A pragmatic learning-based congestion control for the Internet[C]. Proceedings of the Annual Conference of the ACM Special Interest Group on Data Communication. [S.l.]: ACM, 2020: 632–647.
- [21] Dong M, Li Q, Zarchy D, et al. PCC: Re-architecting congestion control for consistent high performance[C]. 12th USENIX Symposium on Networked Systems Design and Implementation (NSDI 15). [S.l.]: USENIX Association, 2015: 395–408.
- [22] Jain R K, Chiu D M W, Hawe W R. A quantitative measure of fairness and discrimination[R]. [S.l.]: Eastern Research Laboratory, Digital Equipment Corporation, 1984.
- [23] Finn C, Abbeel P, Levine S. Model-agnostic meta-learning for fast adaptation of deep networks[C]. International Conference on Machine Learning. [S.l.]: PMLR, 2017: 1126–1135.

附录 A 程序清单

第三章 Swift 拥塞控制算法 Python 实现 (swift-tcp 文件夹)

`tcp_swift.py`: Swift 强化学习智能体核心实现 (约 380 行), 包含 `TcpSwift` 类, 实现多信号融合拥塞检测、差异化拥塞响应、自适应参数调优 (α 动态调整)、融合控制策略 (BDP 估计与窗口调整)、每流独立状态管理等全部核心算法逻辑

`tcp_base.py`: TCP 观测空间基类 (约 140 行), 实现 `TcpEventBased` (11 维事件驱动) 和 `TcpTimeBased` (16 维时间驱动) 两种观测空间解析模式, 负责将 ns-3 传递的原始观测向量解析为语义化的状态参数

`test_swift.py`: Swift 算法测试与运行脚本 (约 120 行), 基于 ns3-gym 框架的强化学习训练/评估入口, 创建 `OpenGym` 环境实例, 为每个 TCP 套接字 (`socketUuid`) 创建独立的 Swift 智能体实例, 实现多流并发控制

第三章 ns-3 网络仿真 C++ 模块 (swift-tcp 文件夹)

`tcp-swift.cc / tcp-swift.h`: `TcpSwift` 拥塞控制类实现, 继承自 `TcpRlBase` 基类, 负责在 ns-3 TCP 协议栈中注册 Swift 算法并创建 `TcpSwiftEnv` 环境实例

`tcp-swift-env.cc / tcp-swift-env.h`: Swift 专用 `OpenGym` 环境类 (约 230 行), 继承自 `TcpGymEnv`, 实现 11 维观测空间的数据采集与传递、ECN 增强的奖励计算 (ECN 惩罚因子)、延迟拥塞窗口应用机制 (`Deferred Cwnd`)

第四章网络仿真实验配置 (swift-tcp 文件夹)

`sim.cc`: ns-3 网络仿真主程序 (约 400 行), 构建哑铃型 (`Dumbbell`) 网络拓扑、配置 TCP 协议参数、设置 `FlowMonitor` 流量监控、配置 `OpenGym` 强化学习接口, 支持通过命令行参数灵活配置 9 种实验场景

`Makefile`: 自动化编译与实验运行脚本, 支持多场景批量实验、对比测试和日志收集

附录 B 攻读专业学位硕士期间撰写的论文

[1] 暂无已发表论文。

附录 C 攻读专业学位硕士期间申请的专利

[1] 暂无已申请专利。

附录 D 攻读专业学位硕士期间参加的科研项目

[1] 暂无参与项目。

致谢

感谢所有对本研究提供帮助和支持的人员。感谢 ns-3 和 OpenGym 开源社区提供的优秀仿真平台和强化学习框架。感谢审稿人提出的宝贵意见和建议。

本研究的完成离不开导师的悉心指导和实验室同学们的帮助。在研究过程中，我们进行了大量的讨论和交流，这些思想碰撞极大地丰富了研究内容。特别感谢实验室提供的计算资源，使得大规模网络仿真实验得以顺利进行。

感谢家人多年来的理解和支持，让我能够专注于科研工作。感谢所有在学习和生活中给予我帮助的老师、同学和朋友们。

最后，感谢计算机网络领域前辈们的开创性工作，正是站在巨人的肩膀上，本研究才得以完成。Van Jacobson、Sally Floyd 等先驱在 TCP 拥塞控制领域的贡献，为后来的研究奠定了坚实的基础。我们希望本研究能够在他们开创的道路上继续前行，为网络拥塞控制技术的发展做出微薄的贡献。